

SCORE

Universeller OCR-Belegerkennungsprozess mit Fuzzy-Logik-basierter Verfahrensauswahl

Version 1.5 · Stand: 12. Juli 2026

Projektname und Akronym

Smart Cost-Optimized Recognition & Evaluation Engine

- S** **Smart** — Das System trifft seine Entscheidungen heuristisch auf Basis gespeicherter Erfahrungswerte und optimiert sich dadurch kontinuierlich selbst.
- C** **Cost-Optimized** — Zentrales Ziel ist die Minimierung der Verarbeitungskosten durch die Wahl zwischen lokalen und Remote-Verfahren sowie zwischen günstigen und teuren LLMs.
- R** **Recognition & Resource** — Steht für die eigentliche OCR-Erkennung sowie für die Auswahl der dafür passenden Ressource (Tool- bzw. Modul-Auswahl).
- E** **Evaluation Engine** — Das System führt vor jeder Verarbeitung eine Vorprüfung und Bewertung durch (Prüfung auf OCR-Tauglichkeit, Gesamtbewertung der Kandidaten) und funktioniert damit wie eine Engine, die fortlaufend Scores berechnet.

INHALTSVERZEICHNIS

1. **Executive Summary**
2. **Ausgangslage**
 1. Hintergrund (Forschungsstand)
3. **Zielsetzung**
 1. Abgrenzung (Out of Scope)
4. **Prozessübersicht**
5. **Funktionale Anforderungen**
6. **Datenmodell**
 1. SCORE-DB (Fallspeicher)
 2. Zeitreihen-DB (Gewichtungs- und Kostenverlauf)
7. **Verarbeitungsmodule und Eskalationslogik**
 1. Grundprinzip und Beispielmodule
 2. Dynamische Eskalation über eine Datenbank-View
 3. Onboarding neuer Verarbeitungsmodule
8. **Algorithmus zur Verfahrensauswahl**
9. **Housekeeping-Prozess (ML-basierte Gewichtsermittlung)**
 1. Datenbereinigungsanalyse (Retention)

2. Administrations-Dashboard (Monitoring)

3. Sicherheitskonzept (Empfehlungen)

10. Nicht-funktionale Anforderungen

11. Offene Punkte / Annahmen

12. Schnittstellenspezifikation (HTTPS-API)

1. Aufruf des Hauptprozesses

2. Beispiel Request-Body

3. Kostenobergrenze und Fallback

4. Beispiel Response-Body

5. HTTP-Statuscodes

6. Rückmelde-Endpunkt

13. Alternative Ähnlichkeitsverfahren (Ausblick)

14. Deployment

1. TLS-Verschlüsselung aller HTTPS-Schnittstellen

2. Docker-Container

3. Betriebsumgebung: Failover- und Skalierungscluster

15. Testszenarien

1. Integrationstest (Einzeldatei-Workflow)

2. Massentest (Volumen- und Regressionstest)

Anhang A: Messgrößen-Katalog (60 Vor-OCR-Metriken)

1. A.1 Praktikable Teilmenge (ohne hohen Aufwand / komplexe Module)

Anhang B: Mockup des Administrations-Dashboards

Anhang C: DuckDB-Schema (Datenmodell)

Executive Summary

Dieses Dokument beschreibt die Anforderungen an einen universellen OCR-Belegerkennungsprozess, der für jeden eingehenden Beleg automatisch das kostengünstigste, erfahrungsgemäss ausreichende Verarbeitungsverfahren bestimmt — vom lokalen Low-Cost-OCR-Modul bis hin zu grossen LLMs.

Kernidee: Vor jeder OCR-Verarbeitung wird die Bilddatei rein statisch analysiert und in einen Vektor standardisierter Messgrössen überführt. Über eine gewichtete Distanzberechnung wird in der SCORE-DB der ähnlichste, erfolgreich verarbeitete Vergleichsfall ermittelt; dessen Verarbeitungsmodul wird als Vorlage für das aktuelle Verfahren übernommen. Eine vom Auftraggeber vorgegebene Kostenobergrenze (`max_kosten`) wird dabei eingehalten; ist dies nicht möglich, greift mindestens das lokale OCR-Verfahren als Fallback.

Da sich Qualität und Kosten der eingesetzten LLMs laufend ändern, sind sowohl die Token-Kosten-Umrechnungsfaktoren als auch die Gewichtung der einzelnen Messgrössen dynamisch konfigurierbar. Ein wöchentlicher Housekeeping-Prozess wertet die SCORE-DB mittels Machine Learning aus und leitet daraus automatisch aktualisierte Gewichtungsfaktoren ab — das System wird dadurch kontinuierlich präziser und kosteneffizienter. Ergänzend wird die Eskalationsreihenfolge der Module nicht mehr fest im Code hinterlegt, sondern über eine Datenbank-View dynamisch aus der SCORE-DB abgeleitet (siehe Kapitel 6.2); über den Übergabeparameter `strategie` (cost/quality) bestimmt der Aufrufer, ob dabei kosten- oder qualitätsoptimiert eskaliert wird, sodass sich beliebig viele Verarbeitungsmodul ohne strukturelle Änderung anbinden lassen.

Der Zugriff erfolgt über eine per TLS verschlüsselte HTTPS-Schnittstelle, die Bildinhalt, Kostenlimite (`max_kosten`), Strategie (`strategie`) und Pflichtfelder entgegennimmt und die erkannten Felder zusammen mit Nachvollziehbarkeitsinformationen (Messgrössen, Referenzdatensatz, gewähltes Verfahren, Kosten) zurückliefert.

Bei starker Skalierung von SCORE könnten die gesammelten Daten (Messgrössen-Vektoren, verwendete Modelle, Erkennungserfolg) potenziell einen eigenen Wert darstellen. Eine kommerzielle Verwertung erscheint fraglich; Interesse könnte jedoch in der Wissenschaft bestehen. Denkbar wäre, die Erkenntnisse zur Qualität der eingesetzten OCR-/LLM-Modelle sowie zur Gewichtung der Messgrössen — analog zu den Dashboard-Auswertungen (Kapitel 8.2) — regelmässig in einem Blog zu veröffentlichen (siehe offene Punkte, Kapitel 10).

Der gewählte Ansatz einer statischen Vor-Analyse zur Vorhersage des OCR-Erfolgs ist durch den aktuellen Forschungsstand gedeckt: Im Forschungsfeld *Document Image Quality Assessment* (DIQA) wird die Vorhersage der OCR-Qualität anhand objektiver Bildmerkmale als eines der wichtigsten Zukunftsthemen eingestuft, und mehrere aktuelle Arbeiten (2024–2025) bestätigen, dass sich der OCR-Erfolg bereits vor der eigentlichen Erkennung mit guter Genauigkeit abschätzen lässt (siehe Kapitel 1.1). Da dieses Forschungsfeld weiterhin dynamisch ist, ist der Messgrössen-Katalog von SCORE bewusst erweiterbar ausgelegt (bis zu 60 Messgrössen vorgesehen, siehe Kapitel 2 und Anhang A).

1. Ausgangslage

Belege wie Rechnungen, Quittungen oder Formulare unterscheiden sich stark in Bildqualität, Layout und Inhalt. Ein einziges, fest verdrahtetes OCR-Verfahren kann dieser Bandbreite nicht gerecht werden:

- Einfache, klare Belege liessen sich günstig mit einem lokalen OCR-Modul verarbeiten.
- Komplexe oder schlecht gescannte Belege brauchen ein leistungsfähigeres, aber teureres LLM-Verfahren.

Es fehlt bisher ein Mechanismus, der pro Beleg automatisch das passende, kostenoptimale Verfahren wählt — basierend auf der Erfahrung aus bereits abgeschlossenen Verarbeitungsfällen.

1.1 Hintergrund (Forschungsstand)

Der SCORE-Ansatz — OCR-Erfolg vor der eigentlichen Erkennung anhand statischer Bildmerkmale abzuschätzen (vgl. FA-1, Anhang A) — ist kein Einzelfall, sondern Teil eines etablierten Forschungsfelds: *Document Image Quality Assessment* (DIQA), auch OCR-Qualitätsvorhersage genannt.

Der Grundgedanke: Statt den OCR-Text zu erzeugen und erst danach mit einer Referenz zu vergleichen, wird bereits aus den Bildeigenschaften abgeschätzt, wie gut die Erkennung ausfallen wird. So lassen sich ungeeignete Belege früh erkennen und gezielt nachbearbeiten, erneut einlesen oder einem stärkeren Verfahren zuweisen — genau das Prinzip hinter der Vor-Analyse und Eskalation in SCORE (Kapitel 3, 6, 7).

Als starke Prädiktoren gelten u. a.:

- Bildschärfe, Kontrast, Bildrauschen, Zeichenhöhe
- Schräglage, perspektivische Verzerrung, Beleuchtungsgleichmässigkeit
- statistische Grössen wie Entropie, Kantendichte, Textdichte

Diese Merkmale bilden zusammen einen Qualitätsvektor — konzeptionell entspricht das dem 60-teiligen Messgrössen-Vektor aus Anhang A. Mehrere Merkmale kombiniert liefern zuverlässigere Vorhersagen als einzelne feste Schwellenwerte (z. B. «mind. 300 dpi») — das deckt sich mit der gewichteten Mehrmerkmals-Distanz in SCORE (Kapitel 7).

Für die Modellbildung kommen typischerweise Random Forests, Gradient Boosting oder neuronale Netze zum Einsatz, mit Zielgrössen wie Character Error Rate (CER) oder Feldgenauigkeit. Das entspricht methodisch dem Housekeeping-Prozess aus Kapitel 8, der ebenfalls per Machine Learning Gewichtungsfaktoren ableitet.

Wichtige Arbeiten dazu:

- Alaei, Bui, Doermann & Pal (2024) — Überblicksarbeit zum Forschungsfeld, benennt OCR-Qualitätsvorhersage als zentrales künftiges Thema.
- Kopytek, Lech & Okarma (2024) — zeigen, dass sich OCR-Leistung aus objektiven Bildqualitätsmassen gut vorhersagen lässt; Beleuchtung, Kontrast und Schärfe sind starke Prädiktoren.
- Gao et al., DeQA-Doc (2025) — nutzt Vision-Language-Modelle zur direkten Qualitätsbewertung, übertrifft klassische Verfahren.
- Ma et al., DocIQ (2025) — neuer Benchmark-Datensatz zur Vereinheitlichung künftiger Forschung.

Fazit für SCORE: Statt jeden Beleg ungeprüft einer teuren OCR-Verarbeitung zuzuführen, schätzt ein Qualitätsmodell zuerst den erwarteten Erfolg ab — und steuert so die Wahl des kosteneffizienten Verfahrens.

Referenzen:

- Alaei, A., Bui, V., Doermann, D., & Pal, U. (2024). Document Image Quality Assessment: A Survey. *ACM Computing Surveys*, 56(2). DOI: 10.1145/3606692.
- Kopytek, M., Lech, P., & Okarma, K. (2024). Application of Binary Image Quality Assessment Methods to Predict the Quality of Optical Character Recognition Results. *Applied Sciences*, 14(22), 10275.
- Gao, J. et al. (2025). DeQA-Doc: Adapting DeQA-Score to Document Image Quality Assessment. arXiv:2507.12796.
- Ma, Z. et al. (2025). DocIQ: A Benchmark Dataset and Feature Fusion Network for Document Image Quality Assessment. arXiv:2509.17012.

2. Zielsetzung

- Automatische Bestimmung eines geeigneten OCR-Verfahrens je Beleg, ohne manuelle Vorabklassifikation.
- Kostenoptimierung durch Eskalation: Wahl des günstigsten Verfahrens, das erfahrungsgemäss für vergleichbare Belege ausreichend war.
- Kontinuierliches Lernen aus der SCORE-DB mit allen bisherigen Verarbeitungen (inkl. Erfolg/Misserfolg und nachträglicher manueller Korrekturen).
- Nachvollziehbarkeit: Zu jedem Verarbeitungsfall ist ersichtlich, welcher historische Fall als Vorlage diente.
- Anpassungsfähigkeit an sich ändernde Kosten- und Qualitätsverhältnisse der eingesetzten LLMs — einschliesslich der Möglichkeit, neue Modelle jederzeit anzubinden.
- Erweiterbarkeit der Messgrössen: Aufgrund der dynamischen Forschungsaktivität im Bereich Document Image Quality Assessment (siehe Kapitel 1.1) können zusätzliche Messgrössen einfach in den Vektor integriert werden — bis zu 60 Messgrössen sind vorgesehen (siehe Anhang A).

2.1 Abgrenzung (Out of Scope)

Nicht Bestandteil des SCORE-Hauptprozesses:

- Die Maske bzw. Benutzeroberfläche zum Prüfen, Korrigieren und Ergänzen der erkannten Belegdaten (manuelle Nachbearbeitung) gehört nicht zum Hauptprozess. Diese Aktivität liegt in der Verantwortung des aufrufenden Systems (z. B. DATEV oder ein vorgelagertes Buchhaltungs-/Erfassungssystem).
- Ebenso erfolgt der Prozess, der den Verarbeitungsfall als «nicht erfolgreiches OCR» kennzeichnet und die entsprechende Folgeverarbeitung (manuelle Nachbearbeitung) anstösst, im aufrufenden System. SCORE stellt hierfür lediglich den Rückmelde-Endpunkt bereit (FA-19, Kapitel 11.6), über den das aufrufende System den Status nach erfolgter Nachbearbeitung an die SCORE-DB zurückmeldet.

Rückmeldung auch aus der Tiefe der Weiterverarbeitung: Die Rückmeldung eines notwendigen Nachbearbeitungsbedarfs muss nicht nur unmittelbar nach der OCR-Erkennung, sondern auch aus tieferliegenden Schritten der Belegweiterverarbeitung erfolgen können. Muss beispielsweise ein Einbuchungsvorschlag tief im DATEV-System nachträglich korrigiert werden, so muss auch dies an SCORE zurückgemeldet werden — denn die Ursache (Root Cause) einer solchen Korrektur kann ein OCR-Fehler sein. Die Herstellung dieses Rückkanals liegt ausserhalb des Verantwortungsbereichs von SCORE und ist im aufrufenden bzw. weiterverarbeitenden System zu leisten. Unter Umständen sind hierfür tiefgehende Analysen von DATEV-Logfiles erforderlich; idealerweise kann stattdessen auf ein Event-Modell von DATEV zurückgegriffen werden. Voraussetzung ist in jedem Fall, dass die SCORE-ID als eindeutige Referenz durch die gesamte Weiterverarbeitung hindurchgereicht werden kann. (*To be verified — die technische Machbarkeit dieser Durchreichung ist noch zu prüfen.*)

SCORE selbst liefert ausschliesslich die erkannten Felder, die Verarbeitungsinfo sowie einen initialen Status (erfolgreich / manuelle Nachbearbeitung erforderlich, siehe Kapitel 7 und 11.4); Prüf-, Korrektur- und Erfassungsoberflächen sowie die fachliche Entscheidung über den endgültigen Verarbeitungsausgang sind nicht Teil dieses Anforderungsdokuments.

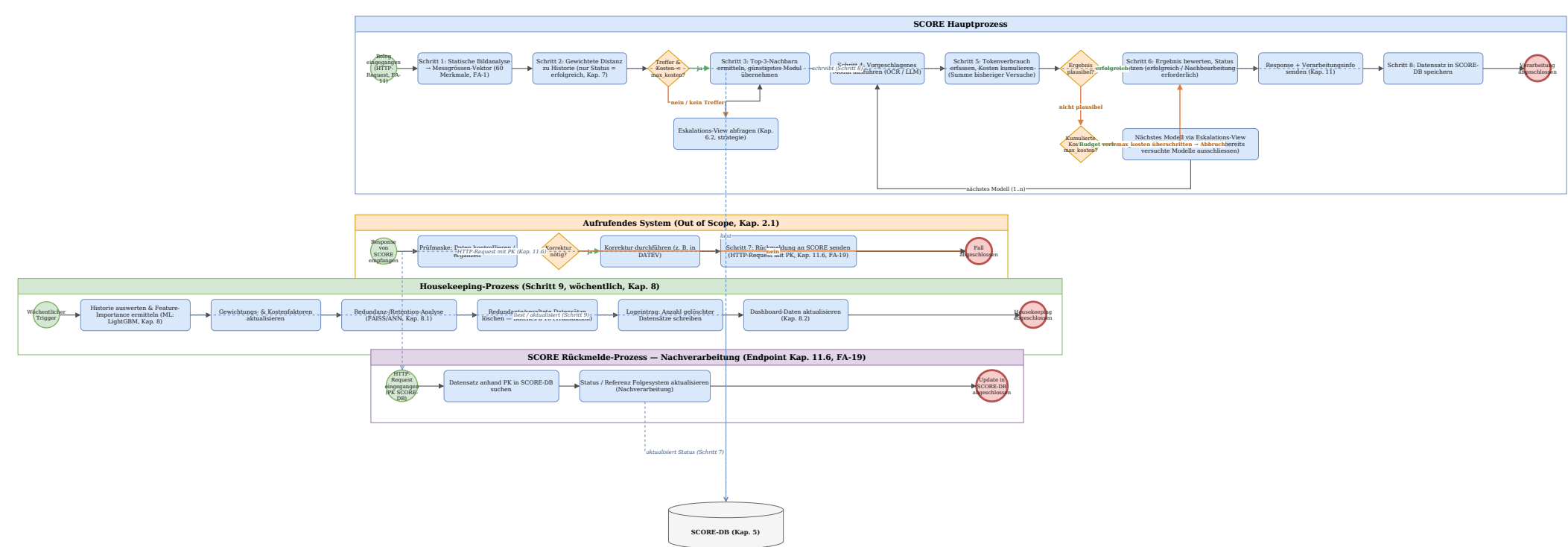
3. Prozessübersicht

Der Verarbeitungsprozess läuft in folgenden Schritten ab (vgl. Prozessdiagramm auf der Folgeseite):

1. Statische Analyse der eingehenden Bilddatei → Ermittlung des Messgrößen-Vektors (ohne OCR-Aufruf).
2. Berechnung der gewichteten Distanz zu allen Datensätzen der SCORE-DB mit Status «erfolgreich» (ohne manuelle Nachbearbeitung).
3. Auswahl des Datensatzes mit der geringsten Distanz; dessen Verarbeitungsmodul wird als Vorschlag übernommen.
4. Ausführung des vorgeschlagenen Verarbeitungsmoduls (z. B. Low-Cost-OCR oder eines von beliebig vielen angebundenen LLM-Modulen, siehe Kapitel 6).
5. Erfassung des Tokenverbrauchs und Kumulierung der Kosten über alle bisherigen Versuche dieses Belegs, anhand der aktuell gültigen, dynamisch konfigurierten Kostenfaktoren.
6. Bewertung des Ergebnisses:
 - Ist das Ergebnis plausibel, wird der Status «erfolgreich» gesetzt und mit Schritt 7 fortgefahren.
 - Ist das Ergebnis nicht plausibel und liegen die kumulierten Kosten noch unter `max_kosten`, wird über die Eskalations-View (Kapitel 6.2, gesteuert durch den Parameter `strategie`) das nächste, noch nicht versuchte Modell ermittelt und der Ablauf ab Schritt 4 wiederholt — so werden bei Bedarf 1 bis n Modelle nacheinander aufgerufen.
 - Überschreiten die kumulierten Kosten `max_kosten`, wird abgebrochen und der Status «manuelle Nachbearbeitung erforderlich» gesetzt.
7. Bei späterer manueller Korrektur (auch in externen Systemen wie DATEV) wird der Status nachträglich aktualisiert.
8. Der abgeschlossene Fall wird selbst Teil der SCORE-DB für künftige Vergleiche.
9. Wöchentlich: Ein Housekeeping-Prozess ermittelt per Machine Learning aktualisierte Gewichtungsfaktoren je Messgröße und Verfahren für die Distanzberechnung und bereinigt die SCORE-DB (Kapitel 8).

SCORE — Prozessdiagramm (BPMN, vereinfacht)

Schrittnummerierung 1-9 gemäß Kapitel 3 (Prozessübersicht)



4. Funktionale Anforderungen

ID	Anforderung	Beschreibung	Priorität
FA-1	Statische Bildanalyse	Beim Eingang eines Belegs wird ohne OCR-Aufruf ein Vektor standardisierter Bild-Messgrößen ermittelt (z. B. Auflösung, Schärfe, Kontrast, Skew, Perspektive — siehe Messgrößen-Katalog in Anhang A, 60 Metriken).	Hoch
FA-2	SCORE-DB	Alle bisherigen Verarbeitungen werden mit Messgrößen-Vektor, verwendetem Verarbeitungsmodul, Tokenverbrauch, Kostenfaktor und Ergebnis-Status persistiert.	Hoch
FA-3	Ähnlichkeitssuche (Fuzzy-Logik)	Für eine neue Datei wird die gewichtete Distanz zu historischen Datensätzen als Summe der gewichteten, quadrierten Differenzen der einzelnen Messgrößen berechnet. Berücksichtigt werden ausschliesslich historische Datensätze mit Status «erfolgreich» und ohne manuelle Nachbearbeitung.	Hoch
FA-4	Verfahrensableitung	Der Datensatz mit der geringsten Distanz wird ausgewählt; dessen Verarbeitungsmodul wird als Vorschlag für die aktuelle Datei übernommen.	Hoch
FA-5	Modulauswahl	Verarbeitungsmodule werden nicht als feste, hart kodierte Stufenfolge hinterlegt, sondern dynamisch aus der SCORE-DB abgeleitet (siehe Kapitel 6). Die Anzahl unterstützter Module ist beliebig erweiterbar; die Auswahl erfolgt kosten- oder qualitätsoptimiert nach dem Eskalationsprinzip aus Kapitel 6.2.	Hoch
FA-6	Ausführung & Tokenmessung	Bei Ausführung des gewählten Verfahrens wird die Anzahl verbrauchter Token erfasst und anhand eines modulabhängigen Kostenfaktors in Kosten umgerechnet.	Hoch
FA-7	Erfolgsstatus	War die OCR-Verarbeitung erfolgreich (Validierung z. B. über Plausibilitäts-/Confidence-Prüfung), wird der Datensatz-Status entsprechend gesetzt und in der Historie fortgeschrieben.	Hoch
FA-8	Nachbearbeitungsstatus	Ist eine manuelle Nachbearbeitung des OCR-Ergebnisses nötig, wird der Status vermerkt — auch wenn diese Nachbearbeitung zeitlich später und in einem externen System (z. B. DATEV) erfolgt.	Hoch
FA-9	Nachträgliche Statusaktualisierung	Das System muss Status-Updates aus nachgelagerten Systemen (z. B. DATEV-Buchungskorrekturen Tage später) nachträglich dem ursprünglichen Verarbeitungsdatensatz zuordnen können.	Mittel
FA-10	Lernender Rückkopplungskreislauf	Jeder neu abgeschlossene Verarbeitungsfall (inkl. finale Status) wird selbst Teil der SCORE-DB und verbessert künftige Fuzzy-Treffer.	Mittel
FA-11	HTTP-Schnittstelle (Trigger)	Der Hauptverarbeitungsprozess wird über einen HTTP-Request gestartet. Der Aufruf erfolgt über HTTPS mit TLS-Verschlüsselung nach aktuellem Standard (siehe Kapitel 13, Deployment) und synchron : Der Aufrufer wartet auf die Antwort des Hauptprozesses und erhält das Ergebnis (erkannte Felder und Verarbeitungsinfo,	Hoch

		FA-13) direkt in der HTTP-Response desselben Requests. Übergeben werden der Bildinhalt, der Parameter <code>max_kosten</code> , der Parameter <code>strategie</code> (Wert <code>cost</code> oder <code>quality</code>); steuert gemäss Kapitel 6.2 die Sortierung der Eskalations-View bei der Ermittlung des nächsten Modells) sowie eine JSON-Struktur mit den Pflichtfeldern, die aus dem Beleg erkannt werden müssen.	
FA-12	Kostenobergrenze / Fallback-Regel	Kann anhand der Fuzzy-Suche kein Verarbeitungsverfahren ermittelt werden, dessen erwartete Kosten unterhalb von <code>max_kosten</code> liegen, wird mindestens das lokale Low-Cost-OCR-Verfahren ausgeführt.	Hoch
FA-13	Rückgabestruktur	Der HTTP-Request liefert als Antwort (a) eine JSON-Struktur mit den erkannten Feldern sowie (b) eine JSON-Struktur mit Informationen zu den ermittelten Messgrössen und dem referenzierten Datensatz in der Datenbank.	Hoch
FA-14	HTTP-Statuscodes	Der Hauptprozess liefert die üblichen HTTP-Returncodes (z. B. 200 bei Erfolg, 4xx bei fehlerhafter Anfrage, 5xx bei Serverfehlern), um den Verarbeitungsstatus für aufrufende Systeme eindeutig erkennbar zu machen.	Mittel
FA-15	Kostenerfassung lokales OCR	Bei Verarbeitung mit dem lokalen (On-Site) OCR-Modul werden die Kosten des Verarbeitungsfalls auf 0 gesetzt, da keine externen Token-Kosten anfallen.	Hoch
FA-16	Dynamische Kostenfaktoren	Die Umrechnungsfaktoren von Token zu Kosten je Verarbeitungsverfahren müssen dynamisch konfigurierbar sein, da sich Preise und Qualität der einzelnen LLMs laufend ändern können.	Hoch
FA-17	Wöchentlicher Housekeeping-Prozess (ML)	Ein wöchentlich laufender Prozess liest die SCORE-DB aus und ermittelt mittels Machine Learning, welche Messgrössen bei welchem Verarbeitungsverfahren welchen Einfluss auf einen erfolgreichen OCR-Ausgang haben. Daraus werden Gewichtungsfaktoren je Messgrösse abgeleitet und der Distanzberechnung (FA-3) zur Verfügung gestellt.	Hoch
FA-19	Rückmelde-Schnittstelle (HTTPS)	Die Rückmeldung einer späteren manuellen Nachbearbeitung (z. B. in DATEV) erfolgt über einen eigenen, TLS-verschlüsselten HTTPS-Request-Prozess, dem ein Identifikator des ursprünglichen Belegs/Datensatzes übergeben wird und der den entsprechenden Datensatz in der SCORE-DB aktualisiert.	Hoch
FA-20	Datenbereinigungsanalyse & automatische Löschung (Retention)	Der Housekeeping-Prozess ermittelt, welche historischen Datensätze gelöscht werden können, ohne den Informationswert der SCORE-DB für die Ähnlichkeitssuche zu mindern, und löscht diese anschliessend automatisch in Transaktionen zu je 10 Datensätzen. Ein Logeintrag mit der Anzahl gelöschter Datensätze wird erzeugt.	Mittel
FA-21	Onboarding neuer Verarbeitungsmodule	Ein neues Verarbeitungsmodul wird als zusätzlicher Branch mit dediziertem Node im Workflow	Mittel

		implementiert. Zur sofortigen Berücksichtigung in Ähnlichkeitssuche und Eskalations-View werden ausgewählte Datensätze eines vergleichbaren Bestandsmodells aus der SCORE-DB dupliziert, der Modulname auf das neue Modul geändert und die Kosten auf 0 gesetzt (siehe Kapitel 6.3).	
FA-22	Administrations-Dashboard	Auf Basis der SCORE-DB und der (append-only geführten) Zeitreihen-DB wird ein Dashboard bereitgestellt, das die Gesamterkennungsrate ohne Nachbearbeitung, die Qualität der einzelnen Verarbeitungsmodule sowie die Predictability der Messgrößen jeweils im Zeitverlauf darstellt (siehe Kapitel 8.2).	Mittel

5. Datenmodell

SCORE nutzt zwei getrennte Datenbanken mit unterschiedlicher Aufgabe:

- die **SCORE-DB** als zentralen Fallspeicher, der zu jedem verarbeiteten Beleg einen Datensatz vorhält (Kapitel 5.1);
- die **Zeitreihen-DB**, in der der Housekeeping-Prozess die je Messgrösse und Modul ermittelten Gewichtungs- und Kostenfaktoren fortlaufend als Zeitreihe ablegt (Kapitel 5.2).

Die SCORE-DB ist die Grundlage für die Ähnlichkeitssuche (Kapitel 7) und die Eskalations-View (Kapitel 6.2); die Zeitreihen-DB liefert die aktuell gültigen Gewichte für die Distanzberechnung und speist zugleich die Predictability-Ansicht des Dashboards (Kapitel 8.2).

5.1 SCORE-DB (Fallspeicher)

Die SCORE-DB speichert je Verarbeitungsfall mindestens folgende Felder:

FELD	BESCHREIBUNG
PK (Primärschlüssel)	Automatisiert generierter, eindeutiger Schlüssel des Datensatzes (z. B. Autoincrement oder UUID).
Zeitstempel	Zeitpunkt der Erstellung des Datensatzes (Beleg-Eingang / Verarbeitungsstart).
Messgrössen-Vektor (60 Werte)	Normierte Werte 0-100 je Metrik gemäss Messgrössen-Katalog (Anhang A).
Verwendetes Modul	Name des eingesetzten Verarbeitungsmoduls (z. B. Low-Cost-OCR, LLM klein, LLM mittel, LLM gross oder ein beliebiges weiteres, gemäss Kapitel 6 angebundenes Modul). Das Feld ist nicht auf eine feste Werteliste beschränkt.
Token-Anzahl	Für den Verarbeitungslauf verbrauchte Token.
Kostenfaktor / Kosten gesamt	Modulabhängiger, dynamisch konfigurierbarer Faktor bzw. daraus berechnete Gesamtkosten.
Status Verarbeitung	erfolgreich manuelle Nachbearbeitung erforderlich in Bearbeitung.
Zeitstempel Statusänderung	Zeitpunkt, an dem der Status zuletzt aktualisiert wurde (kann später liegen als der Eingang).
Referenz Folgesystem	z. B. DATEV-Belegnummer, falls Nachbearbeitung dort erfolgte.
Distanz zum Vorschlag (Fuzzy-Score)	Gewichteter Ähnlichkeitswert zum verwendeten Referenzdatensatz, zur Nachvollziehbarkeit.
Daten (Datentyp JSON)	JSON-Feld mit den bei der Verarbeitung ermittelten Daten (erkannte Felder) sowie zugehörigen Log-Informationen des Verarbeitungslaufs.

5.2 Zeitreihen-DB (Gewichtungs- und Kostenverlauf)

Getrennt von der SCORE-DB wird eine Zeitreihen-DB geführt, die vom Housekeeping-Prozess aktualisiert wird und die aktuell gültigen Gewichtungs- und Kostenfaktoren je Messgrösse und Verfahren enthält:

FELD	BESCHREIBUNG
Messgrösse-ID	Referenz auf eine der 60 Messgrössen des Katalogs.
Verarbeitungsmodul	Freier Modulname; Gewichte gelten je Modul separat. Die Anzahl unterstützter Module ist nicht begrenzt (siehe Kapitel 6).
Gewichtungsfaktor w_i	Vom Housekeeping-Prozess ermittelter Faktor zur Skalierung der jeweiligen Messgrösse in der Distanzformel.
Kostenfaktor je Token	Aktuell gültiger, dynamisch konfigurierbarer Umrechnungsfaktor Token → Kosten für dieses Modul.
Zeitstempel letzte Aktualisierung	Zeitpunkt des Housekeeping-Laufs, der diesen Datensatz geschrieben hat.

Damit sich der Verlauf der Gewichtungsfaktoren über mehrere Housekeeping-Läufe auswerten lässt (siehe Kapitel 8.2, Predictability-Ansicht des Dashboards), wird die Zeitreihen-DB *append-only* geführt: Jeder Housekeeping-Lauf schreibt je Messgrösse und Modul einen neuen, mit Zeitstempel versehenen Datensatz, statt den vorherigen Wert zu überschreiben. Für die operative Nutzung in Kapitel 7 wird stets der jeweils aktuellste Datensatz je Messgrösse/Modul herangezogen.

Eine konkrete, lauffähige Umsetzung dieses Datenmodells als DuckDB-Schema — inklusive beider Tabellen, der Fuzzy-Distanz-View mit gewichteter Distanzberechnung (Kapitel 7), der Trigger-Ersatzmechanik für die Zeitstempel, einer Export-View mit inhaltlich benannten Messwert-Spalten sowie Benutzer-/Rechtekonzept und Testdaten-Prozedur — findet sich in [Anhang C](#).

6. Verarbeitungsmodule und Eskalationslogik

6.1 Grundprinzip und Beispielmodule

Das System unterstützt mehrere Verarbeitungsstufen mit steigenden Kosten und steigender Leistungsfähigkeit. Die folgenden Module dienen als illustrative Beispiele; die tatsächlich verfügbare Modulmenge ist — wie in Kapitel 6.2 beschrieben — nicht fest vorgegeben, sondern beliebig erweiterbar:

- **Low-Cost On-Site-OCR-Modul** — lokal, günstigste Stufe, für einfache, klare Belege. Die Kosten werden für diese Stufe gemäss FA-15 stets auf 0 gesetzt.
- **LLM klein** — moderater Kostenaufwand, für Belege mit leichten Qualitäts- oder Layoutproblemen.
- **LLM mittel** — höhere Kosten, für komplexere oder schwierigere Belege.
- **LLM gross** — höchste Kosten, für Belege mit stark abweichenden oder schlechten Messgrössen bzw. wenn kleinere Module in vergleichbaren Fällen nicht erfolgreich waren.

6.2 Dynamische Eskalation über eine Datenbank-View

Die Eskalationshierarchie zwischen den Verarbeitungsmodulen wird nicht fest im Code oder als starre Stufenfolge konfiguriert. Stattdessen wird sie über eine Datenbank-View gelöst, die je Modul die historische Kosten- und Erkennungsleistung aggregiert.

Gesteuert wird dies über einen neuen Übergabeparameter `strategie` des Hauptprozesses (siehe FA-11 und Kapitel 11.1/11.2) mit den möglichen Werten `cost` oder `quality`. Dieser Parameter bestimmt, welche der beiden Views zur Ermittlung des nächsten Modells (Eskalations-/Fallback-Kandidat) herangezogen wird:

- Die View gruppiert alle relevanten Datensätze der SCORE-DB je `verwendetes_modul` und berechnet pro Modul die durchschnittlichen Kosten sowie die Erkennungsrate.
- *Strategie Cost* (`strategie = "cost"`): Die View sortiert nach den durchschnittlichen Kosten (`avg_kosten`) aufsteigend — das günstigste Modul steht an erster Stelle.
- *Strategie Quality* (`strategie = "quality"`): Die Erkennungsrate («quality») wird je Modul gebildet als Anzahl erfolgreicher Durchläufe mit diesem Modul geteilt durch die Gesamtzahl der Datensätze mit diesem Modell — jeweils bezogen auf die Datensätze der letzten 100 Tage. Die View sortiert absteigend nach dieser Erkennungsrate — das zuverlässigste Modul steht an erster Stelle.
- Das System wählt anschliessend das Modul aus dem **ersten Datensatz** der jeweils aktiven View als Eskalations- bzw. Fallback-Kandidaten.

Durch diesen View-basierten Ansatz entfällt eine fest verdrahtete Eskalationsfolge; es können dadurch beliebig viele Modelle unterstützt werden, ohne dass Code oder Konfiguration strukturell angepasst werden müssen.

```

-- Strategie: cost (strategie = "cost")
CREATE VIEW v_modell_ranking_cost AS
SELECT
    verwendetes_modul      AS modell,
    AVG(kosten_gesamt)     AS avg_kosten
FROM score_db
GROUP BY verwendetes_modul
ORDER BY avg_kosten ASC;

-- Strategie: quality (strategie = "quality")
-- Erkennungsrate = Anzahl erfolgreicher Durchläufe je Modell
--                  / Gesamtzahl Datensätze je Modell,
--                  jeweils eingeschränkt auf die letzten 100 Tage.
CREATE VIEW v_modell_ranking_quality AS
SELECT
    verwendetes_modul AS modell,
    SUM(CASE WHEN status = 'erfolgreich' THEN 1 ELSE 0 END)
      / COUNT(*) * 100                                AS erkennungsrate
FROM score_db
WHERE zeitstempel >= NOW() - INTERVAL '100 days'
GROUP BY verwendetes_modul
ORDER BY erkennungsrate DESC;

```

Hinweis zur Konsistenz: Massgeblich für die Auswahl ist stets der *erste* Datensatz der jeweils gemäss **strategie** aktiven View — bei «cost» somit das Modul mit den geringsten Durchschnittskosten, bei «quality» das Modul mit der höchsten Erkennungsrate der letzten 100 Tage. Dies steht im Einklang mit der in Kapitel 2 formulierten Zielsetzung, je nach Vorgabe entweder das günstigste oder das zuverlässigste Verfahren zu wählen.

Diese modellbasierte Eskalations-View ergänzt die fallbezogene Ähnlichkeitssuche aus Kapitel 7: Während Kapitel 7 für jeden einzelnen Beleg den ähnlichsten historischen Fall heranzieht, liefert die Eskalations-View eine globale, jederzeit aktuelle Rangfolge aller verfügbaren Module gemäss der gewählten Strategie. Sie kommt insbesondere in der Kaltstart-Situation (siehe Kapitel 10) sowie als Fallback zur Anwendung, wenn im Umkreis der Ähnlichkeitssuche keine ausreichende Anzahl historischer Treffer vorhanden ist.

6.3 Onboarding neuer Verarbeitungsmodule

Da die Anzahl unterstützter Module nicht begrenzt ist, können jederzeit weitere Verarbeitungsmodule ergänzt werden. Neue Module bedingen naturgemäss einen neuen Branch im Workflow mit einem spezifischen, für das jeweilige Modell zuständigen Node.

Damit ein neu angebundenes Modul unmittelbar von der Ähnlichkeitssuche (Kapitel 7) und der Eskalations-View (Kapitel 6.2) gefunden wird, obwohl noch keine eigene Historie existiert (Kaltstart), wird eine definierte Anzahl an Datensätzen eines vergleichbaren Bestandsmodells in der SCORE-DB dupliziert:

- Der Modulname der kopierten Datensätze wird durch den Namen des neuen Modells ersetzt (entsprechend dem neuen Branch im Workflow).
- Die Kosten dieser kopierten Datensätze werden auf 0 gesetzt.
- Dadurch erscheint das neue Modell in der Kosten-View (6.2) vorübergehend als besonders günstig und wird bevorzugt zur Erprobung ausgewählt — es «findet» sich damit selbst in den Vergleichsmechanismen wieder, bis ausreichend echte Verarbeitungsergebnisse vorliegen.

Diese Seeding-Aktivität lässt sich als einfaches SQL-Statement implementieren und bei Bedarf direkt auf der Datenbank-Engine ausführen:

```
INSERT INTO score_db (
    zeitstempel, messgroessen_vektor, verwendetes_modul, token_anzahl,
    kostenfaktor, kosten_gesamt, status, zeitstempel_statusaenderung,
    referenz_folgesystem, fuzzy_distanz, daten
)
SELECT
    NOW(), messgroessen_vektor, 'LLM-neues-modell-x', token_anzahl,
    0, 0, status, zeitstempel_statusaenderung,
    referenz_folgesystem, fuzzy_distanz, daten
FROM score_db
WHERE verwendetes_modul = 'LLM mittel'
    AND status = 'erfolgreich'
ORDER BY RANDOM()
LIMIT 50;
```

Sobald das neue Modell über eine ausreichende Anzahl eigener, realer Verarbeitungsfälle verfügt, sollten die künstlich erzeugten Seed-Datensätze markiert oder entfernt werden, damit die Kostenkennzahlen der View ausschliesslich auf tatsächlichen Ergebnissen beruhen.

7. Algorithmus zur Verfahrensauswahl

Die Ermittlung des Verarbeitungsvorschlags erfolgt in der Datenbank (Datenview) nach folgendem, deterministischem Verfahren:

1. **Filterung:** Es werden ausschliesslich Datensätze der SCORE-DB mit Status «erfolgreich» berücksichtigt, bei denen keine manuelle Nachbearbeitung erforderlich war.
2. **Distanzberechnung** je verbleibendem Datensatz: Distanz = Summe über alle Messgrößen der mit dem Gewichtungsfaktor w_i skalierten, quadrierten Differenz zwischen dem Wert des historischen Datensatzes und dem Wert des aktuellen Durchlaufs ($\sum w_i \times (m_{\text{historisch},i} - m_{\text{aktuell},i})^2$).
3. **Selektion:** Der Datensatz mit der geringsten Distanz wird ermittelt (nächster Nachbar). Es wird genau ein Datensatz zurückgeliefert.
4. **Übernahme:** Das im selektierten Datensatz hinterlegte Verarbeitungsmodul wird als Vorschlag für den aktuellen Durchlauf verwendet (vorbehaltlich der Kostenobergrenze `max_kosten` gemäss FA-12).
5. **Eskalation bei Misserfolg:** Liefert das gewählte Modul kein plausibles Ergebnis, wird — solange die kumulierten Kosten unter `max_kosten` liegen — über die Eskalations-View (Kapitel 6.2, Parameter `strategie`) das nächste, noch nicht versuchte Modell bestimmt und die Verarbeitung wiederholt (vgl. Kapitel 3, Schritt 6).

Hinweis: Es wird bewusst nur der ähnlichste Datensatz herangezogen und dessen Verfahren übernommen. Der Kostenaspekt wird dadurch nicht vernachlässigt: Die laufende Kostenoptimierung erfolgt über die Eskalationslogik (Kapitel 6.2, Parameter `strategie`) sowie dadurch, dass Verarbeitungen mit dem lokalen OCR-Modul stets mit Kosten = 0 geführt werden (FA-15). Die Gewichtungsfaktoren w_i sind je Verarbeitungsmodul separat hinterlegt, da eine Messgrösse den Erfolg unterschiedlicher Verfahren unterschiedlich stark beeinflussen kann.

7.1 Alternative Ähnlichkeitsmethoden in DuckDB

Die oben beschriebene gewichtete euklidische Distanz (L2) ist die Standardmethode. DuckDB (Kapitel 13.2) bietet jedoch weitere Ähnlichkeitsmethoden, die sich mit geringem Aufwand umsetzen und — analog zum Parameter `strategie` — als wählbare Varianten anbieten lassen. Sie sind für spätere Ausbaustufen vorgesehen und hier als Optionen dokumentiert.

Weitere Distanz- und Ähnlichkeitsmasse (Bordmittel)

- **Manhattan-Distanz (L1):** Summe der absoluten Differenzen statt der quadrierten Differenzen. Robuster gegenüber einzelnen stark abweichenden Messgrößen (Ausreissern), da grosse Abweichungen nicht quadratisch verstärkt werden.
- **Cosinus-Ähnlichkeit:** misst den Winkel zwischen Anfrage- und Referenzvektor statt der absoluten Differenz. Sinnvoll, wenn das *Muster* der Messgrößen wichtiger ist als deren absolute Höhe (z. B. wenn ein Beleg dieselbe Charakteristik, aber durchgängig etwas andere Beleuchtung aufweist). In DuckDB direkt über `list_cosine_similarity()` verfügbar.
- **Tschebyschow-Distanz (L ∞):** stellt auf die einzelne am stärksten abweichende Messgrösse ab (Maximum der absoluten Differenzen). Nützlich, wenn bereits eine einzige gravierend abweichende Eigenschaft einen Referenzfall unbrauchbar macht.

Deklariert man den `messgroessen_vektor` als Array fester Länge (`DOUBLE[60]` statt `DOUBLE[]`), lassen sich zudem die eingebauten Array-Funktionen `array_distance()` (euklidisch), `array_cosine_similarity()` und `array_inner_product()` direkt nutzen — kürzer und schneller als die manuelle Summenbildung. Die gewichtete Variante bleibt durch Vorab-Skalierung jeder Messgrösse mit $\sqrt{w_i}$ erhalten (derselbe Trick wie in Kapitel 8.1).

Beschleunigte Suche über die VSS-Extension

Für grosse Datenmengen (Größenordnung 1 Mio. Datensätze) bietet DuckDB die offizielle *VSS-Extension* (Vector Similarity Search) mit einem HNSW-Index auf Array-Spalten. Dies ist dasselbe Approximate-Nearest-Neighbor-Verfahren, das in Kapitel 8.1 für die Redundanzanalyse vorgesehen ist — hier jedoch direkt in DuckDB, ohne separaten Python-Dienst. Die Top-Treffer-Suche der Ähnlichkeitssuche wird damit von linearem auf sub-linearen Aufwand beschleunigt. Da sich der HNSW-Index über die $\sqrt{w_i}$ -skalierten Vektoren legen lässt, bleibt die gewichtete SCORE-Distanz erhalten. Perspektivisch könnte die VSS-Extension damit sowohl die Ähnlichkeitssuche dieses Kapitels als auch die Redundanzanalyse (Kapitel 8.1) in einer Komponente abdecken.

Diese Methoden sind optionale Erweiterungen; die verbindliche Festlegung, welche Ähnlichkeitsmasse produktiv angeboten werden (und ob analog zu `strategie` ein wählbarer Parameter eingeführt wird), erfolgt gemäss Kapitel 10 (offene Punkte). Weitergehende, rechenintensivere Verfahren (Mahalanobis-Distanz, Clustering) sind in Kapitel 12 beschrieben.

8. Housekeeping-Prozess (ML-basierte Gewichtsermittlung)

Um die Gewichtungs- und Kostenfaktoren an sich ändernde LLM-Qualität und -Preise anzupassen, wird ein wöchentlich laufender Housekeeping-Prozess eingerichtet.

- **Turnus:** wöchentlich, ausserhalb der Hauptverarbeitungszeiten.
- **Input:** sämtliche Datensätze der SCORE-DB inkl. Messgrössen-Vektor, verwendetem Verfahren und finalem Status (erfolgreich / manuelle Nachbearbeitung).
- **Verfahren:** ein Machine-Learning-Modell auf tabellarischen Daten (Feature-Importance aus einem Klassifikationsmodell; empfohlen: LightGBM oder vergleichbares Gradient-Boosting-Verfahren, siehe Empfehlung unten) ermittelt je Verarbeitungsmodul, welche Messgrössen den grössten Einfluss auf einen erfolgreichen OCR-Ausgang haben.
- **Output:** aktualisierte Gewichtungsfaktoren w_i je Messgrösse und Verfahren, die in die Zeitreihen-DB (Kapitel 5.2) geschrieben werden.
- **Wirkung:** die neuen Gewichte werden ab dem nächsten Hauptprozess-Lauf automatisch in der Distanzberechnung (Kapitel 7) verwendet, ohne Codeänderung oder manuellen Eingriff.
- Zusätzlich aktualisiert der Housekeeping-Prozess die Token-Kosten-Umrechnungsfaktoren je Verfahren gemäss FA-16, sofern neue Preisinformationen vorliegen.

Damit passt sich das System kontinuierlich an: Verbessert sich z. B. die Qualität eines LLM-klein-Modells, spiegelt sich dies in den Folgewochen in mehr erfolgreichen Treffern für dieses Modul in der SCORE-DB wider — und damit in einer höheren Wahrscheinlichkeit, dass es als kostengünstige Alternative vorgeschlagen wird.

Predictability der Messgrössen: Ermittlung im Housekeeping-Prozess

Die **Predictability** einer Messgrösse gibt an, wie stark diese einzelne Messgrösse statistisch mit dem tatsächlichen OCR-Erfolg zusammenhängt bzw. wie stark sie zu dessen Vorhersage beiträgt. Operationalisiert wird sie über den Feature-Importance-Wert, den der Housekeeping-Prozess je Messgrösse und Verarbeitungsmodul berechnet und als Gewichtungsfaktor w_i in der Zeitreihen-DB (Kapitel 5.2) hinterlegt.

Kein LLM, sondern klassisches Machine Learning auf tabellarischen Daten: Die Datensätze der SCORE-DB werden für diese Berechnung nicht an ein Sprachmodell (LLM) übergeben. Jeder Datensatz liefert lediglich einen Vektor aus 60 numerischen Messwerten plus ein binäres Zielmerkmal (Status «erfolgreich» ja/nein) — ein klassisches, gut erforschtes Klassifikationsproblem auf strukturierten/tabellarischen Daten. Für diese Art von Problem sind Gradient-Boosting- bzw. Random-Forest-Verfahren nicht nur ausreichend, sondern LLMs sowohl fachlich unpassend (LLMs sind für Sprach- und Textaufgaben optimiert, nicht für die statistische Gewichtung numerischer Merkmale) als auch deutlich unwirtschaftlicher: Die Kosten eines LLM-Aufrufs skalieren mit der Menge der übergebenen Daten (Token-Preis), während spezialisierte tabellarische ML-Bibliotheken lokal auf Standard-Hardware laufen und keine laufenden API-Kosten verursachen.

Konkrete Empfehlung für die Größenordnung von SCORE (ca. 1 Million Datensätze in der SCORE-DB): Für ein Trainingsset dieser Grösse (rund 1 Mio. Zeilen $\times$ 60 Merkmale) ist **LightGBM** (Gradient Boosted Decision Trees, histogrammbasiert) die kosteneffizienteste Wahl: Das Training läuft auf einem einzelnen CPU-Kern typischerweise im Sekunden- bis niedrigen Minutenbereich, benötigt keine GPU, ist speichereffizient und liefert die benötigte Feature-Importance direkt als Nebenprodukt des Trainings. Alternativen sind XGBoost (vergleichbare Genauigkeit, etwas höherer

Rechenaufwand) oder ein einfacherer Random-Forest-Klassifikator (z. B. scikit-learn), der bei dieser Datenmenge ebenfalls praktikabel, aber typischerweise langsamer und speicherintensiver als LightGBM ist. Der wöchentliche Housekeeping-Lauf lässt sich damit als einfacher, geplanter Batch-Job auf Standard-Compute umsetzen — im Einklang mit dem «Cost-Optimized»-Grundsatz von SCORE selbst (siehe Projektname und Akronym).

Das Verfahren läuft wöchentlich erneut, da sich die zugrunde liegenden Muster mit der Zeit verändern können (z. B. andere Aufnahmegeräte, neue Belegtypen). Die Zeitreihe der wöchentlichen `w_i`-Werte je Messgrösse ist im Administrations-Dashboard einsehbar (Kapitel 8.2, Panel 3): Ein deutlicher Rückgang über mehrere Läufe zeigt an, dass eine Messgrösse an Aussagekraft verliert, und ist ein Signal an den Administrator, den Messgrössen-Katalog (Anhang A) zu überprüfen.

Tooling-Vergleich für ca. 1 Million Datensätze

OPTION	EIGNUNG BEI 1 MIO. DATENSÄTZEN	BEWERTUNG
LightGBM (Python, <code>lightgbm</code> -Package)	Sekunden bis wenige Minuten, 1 CPU-Kern, keine GPU nötig	Empfohlen
XGBoost (Python)	ähnlich gut, etwas mehr Rechenzeit	Gute Alternative
scikit-learn RandomForest (Python)	funktioniert, aber spürbar langsamer und speicherhungriger	Nur falls Interpretierbarkeit wichtiger als Tempo
Reines JavaScript (z. B. <code>ml-random-forest</code>)	bei 1 Mio. Zeilen deutlich langsamer, mehr Speicherbedarf, weniger ausgereift	Nicht für Produktion in dieser Grössenordnung
TensorFlow.js	kein natives Gradient-Boosting-Verfahren; für dieses Problem nicht das passende Werkzeug	Ungeeignet

Eine direkte Umsetzung in reinem JavaScript ist bei dieser Datenmenge nicht empfehlenswert: LightGBM und XGBoost verdanken ihre Performance einer in C/C++ implementierten, histogrammbasierten und mehrfädigen (multithreaded) Berechnung — genau das fehlt vergleichbaren JavaScript-Bibliotheken, die in der Regel single-threaded und interpretiert laufen und für diese Grössenordnung nicht auf Produktionsreife ausgelegt sind. Node-Bindings für LightGBM existieren zwar (z. B. `node-lightgbm`), sind jedoch deutlich weniger ausgereift und gepflegt als das offizielle Python-Package.

Integrationsvorschlag in den n8n-Workflow

- Der Housekeeping-Workflow selbst bleibt wie bisher in n8n/JavaScript umgesetzt.
- Der eigentliche ML-Rechenschritt läuft als **eigenständiger, kleiner Python-Dienst** (z. B. ein einfaches Skript oder ein schlanker FastAPI-Endpunkt mit `lightgbm`), den der n8n-Workflow über einen **HTTP-Request-Node** oder einen **Execute-Command-Node** aufruft.
- **Input** an den Python-Dienst: Export der SCORE-DB-Daten (Messgrössen-Vektor + Status) als CSV/Parquet oder direkter Datenbank-Read.
- **Output** an den Workflow zurück: die berechneten Feature-Importance-Werte je Messgrösse, die der Workflow anschliessend in die Zeitreihen-DB (Kapitel 5.2) schreibt.

Damit verbleibt der übrige Housekeeping-Prozess in der gewohnten n8n/JavaScript-Umgebung; lediglich der rechenintensive ML-Schritt läuft dort, wo er technisch und wirtschaftlich am effizientesten ist.

8.1 Datenbereinigungsanalyse (Retention)

Da die SCORE-DB kontinuierlich wächst, prüft der Housekeeping-Prozess zusätzlich, welche Datensätze gefahrlos gelöscht werden können, ohne die Qualität der Ähnlichkeitssuche zu verschlechtern.

- **Kriterium Redundanz:** Datensätze, die im Messgrößen-Raum sehr nahe an zahlreichen anderen, jüngeren erfolgreichen Datensätzen liegen, tragen wenig zusätzlichen Informationswert bei.
- **Kriterium Alter/Relevanz:** sehr alte Datensätze, deren Verarbeitungsmodul oder Kostenfaktor nicht mehr aktiv genutzt bzw. längst durch neuere Modul- oder Preisversionen abgelöst ist, verlieren an Aussagekraft für aktuelle Kostenentscheide.
- **Kriterium Abdeckung:** Datensätze aus Bereichen des Messgrößen-Raums, die durch ausreichend viele andere Datensätze bereits gut abgedeckt sind, können reduziert werden, ohne Lücken im Vergleichsraum zu erzeugen.
- **Output:** Die als redundant bzw. veraltet identifizierten Datensätze werden direkt und automatisch gelöscht — ohne separaten Freigabeprozess und ohne E-Mail-Versand. Um die Live-Tabelle nicht zu blockieren, erfolgt die Löschung in kleinen Transaktionen von jeweils 10 Datensätzen. Nach Abschluss des Housekeeping-Laufs wird ein Logeintrag mit der Gesamtanzahl der gelöschten Datensätze geschrieben; dieser ist im Administrations-Dashboard (Kapitel 8.2) einsehbar.

Löschmechanik: Batches à 10 Datensätze

Die Löschung erfolgt nicht als eine einzelne Grosstransaktion über potenziell tausende Datensätze, sondern in vielen kleinen Schritten zu je 10 Datensätzen. Das hat zwei Gründe: Erstens bleiben die Sperrzeiten auf der SCORE-DB pro Transaktion minimal, sodass der parallel laufende Hauptprozess (Kapitel 3, Ähnlichkeitssuche) nicht spürbar beeinträchtigt wird. Zweitens lässt sich der Vorgang bei einem Abbruch (z. B. Verbindungsfehler) sauber ab dem letzten abgeschlossenen 10er-Batch fortsetzen, ohne den gesamten Löschlauf wiederholen zu müssen.

```
-- Beispielhafte Batch-Löschung (wiederholt ausgeführt, bis keine Kandidaten mehr übrig sind)
DELETE FROM score_db
WHERE pk IN (
    SELECT pk FROM loeschkandidaten
    ORDER BY pk
    LIMIT 10
);
```

Nach jedem vollständigen Housekeeping-Lauf wird ein einzelner zusammenfassender Logeintrag erzeugt, z. B.:

```
{
  "zeitstempel": "2026-07-18T02:00:00Z",
  "housekeeping_lauf_id": "hk-2026-07-18",
  "geloeschte_datensaetze": 4230,
  "anzahl_batches": 423
}
```

Technische Umsetzung der Redundanz-Erkennung bei ca. 1 Million Datensätzen

Für das Kriterium Redundanz wird derselbe gewichtete Abstand wie in der Ähnlichkeitssuche (Kapitel 7) verwendet: Skaliert man jede Messgrösse vorab mit $\sqrt{w_i}$, entspricht ein gewöhnlicher euklidischer (L2-)Abstand exakt der gewichteten SCORE-Fuzzy-Distanz. Dadurch lassen sich für die Redundanzprüfung bewährte, hochoptimierte Standard-Bibliotheken einsetzen, statt eine Eigenlösung zu entwickeln.

Ein paarweiser Vergleich aller Datensätze (bei 1 Mio. Datensätzen $\approx 5 \times 10^{11}$ Paare) ist nicht praktikabel. Empfohlen wird stattdessen eine Approximate-Nearest-Neighbor-Suche (ANN):

ANSATZ	EIGNUNG BEI 1 MIO. DATENSÄTZEN	BEWERTUNG
FAISS (Facebook AI Similarity Search, Python/C++)	sub-lineare Suche auch bei Millionen Vektoren, läuft auf CPU, kostenlos/Open Source	Empfohlen
HNSW (z. B. über <code>hnswlib</code>)	ähnlich performant, etwas einfacher zu integrieren	Gute Alternative
MiniBatchKMeans-Clustering (scikit-learn)	einfacher umzusetzen, gröber; reicht für die Coverage-Entscheidung oft aus	Pragmatische Alternative
Brute-Force-Paarvergleich	bei 1 Mio. Datensätzen nicht praktikabel	Ungeeignet

Konkretes Vorgehen je Housekeeping-Lauf:

1. Alle Datensätze mit Status «erfolgreich» werden geladen und ihre Messgrössen mit $\sqrt{w_i}$ skaliert.
2. Über den skalierten Vektoren wird ein FAISS-Index (z. B. HNSW) aufgebaut; für jeden Datensatz werden die k nächsten Nachbarn ermittelt (z. B. $k=10$).
3. Datensätze mit sehr geringem Abstand zu vielen Nachbarn (hohe lokale Dichte) gelten als redundant; von ihnen wird nur eine Auswahl (z. B. der/die jüngste(n)) behalten, die übrigen werden zur Löschung vorgeschlagen — in Kombination mit dem Kriterium Alter/Relevanz.
4. Datensätze in dünn besiedelten Bereichen des Messgrössen-Raums (wenige oder keine nahen Nachbarn) bleiben unabhängig vom Alter erhalten, da sie einzigartige Fälle abdecken (Kriterium Abdeckung).

Die ANN-Suche lässt sich im selben Python-Microservice umsetzen, der bereits für die Predictability-Berechnung (LightGBM, siehe oben) vorgesehen ist — z. B. durch Ergänzung des Pakets `faiss-cpu`. Ein separates System ist dafür nicht erforderlich.

8.2 Administrations-Dashboard (Monitoring)

Auf Basis der SCORE-DB (Kapitel 5.1) und der Zeitreihen-DB (Kapitel 5.2) wird ein Dashboard bereitgestellt. Es unterstützt den Administrator dabei, frühzeitig zu erkennen, wann ein Verarbeitungsmodul (LLM) ausgetauscht werden sollte (siehe Kapitel 6.3) und ob der bestehende Messgrössen-Katalog (Anhang A) noch aussagekräftig ist.

Im Kopfbereich zeigt das Dashboard neben den Qualitäts-Kennzahlen zwei Bestandskennzahlen der SCORE-DB: die **Gesamtanzahl der Datensätze** (inkl. Zu- und Abgang der laufenden Woche, letzterer durch die Housekeeping-Löschung gemäss

Kapitel 8.1) sowie das **durchschnittliche Alter der Datensätze**. Zusammen erlauben sie eine schnelle Einschätzung, ob die Vergleichsbasis wächst, überaltert oder durch die Retention-Löschung zu stark schrumpft.

```
-- Bestandskennzahlen der SCORE-DB
SELECT
    COUNT(*)                                AS anzahl_datensaetze,
    AVG(EXTRACT(EPOCH FROM (NOW() - zeitstempel))
        / 86400)                            AS avg_alter_tage
FROM score_db;
```

Darüber hinaus zeigt das Dashboard drei Zeitverlaufs-Ansichten:

1. Gesamterkennungsrate ohne Nachbearbeitung im Zeitverlauf

Anteil der Fälle mit Status «erfolgreich» (ohne manuelle Nachbearbeitung) an allen Verarbeitungsfällen, aggregiert je Zeitperiode (z. B. Woche). Ein anhaltender Rückgang dieser Quote — unabhängig vom eingesetzten Modul — deutet darauf hin, dass sich die Belegpopulation oder die Datenbasis grundsätzlich verändert hat.

```
SELECT
    DATE_TRUNC('week', zeitstempel)          AS periode,
    SUM(CASE WHEN status = 'erfolgreich' THEN 1 ELSE 0 END)
        / COUNT(*) * 100                    AS erkennungsrate_gesamt
FROM score_db
GROUP BY periode
ORDER BY periode;
```

2. Qualität der einzelnen Module im Zeitverlauf

Wie Ansicht 1, jedoch je Verarbeitungsmodul (analog zur Eskalations-View aus Kapitel 6.2, hier jedoch zeitlich aufgelöst statt als Gesamtdurchschnitt). Dargestellt als Liniendiagramm mit einer Linie je Modul. Sinkt die Erkennungsrate eines einzelnen Moduls über mehrere Perioden deutlich — z. B. weil ein LLM-Anbieter sein Modell ändert —, ist dies für den Administrator ein konkreter Hinweis, dieses Modul zu überprüfen oder gemäss dem Onboarding-Prozess (Kapitel 6.3, FA-21) durch ein alternatives Modell zu ersetzen.

```
SELECT
    DATE_TRUNC('week', zeitstempel)          AS periode,
    verwendetes_modul                       AS modell,
    SUM(CASE WHEN status = 'erfolgreich' THEN 1 ELSE 0 END)
        / COUNT(*) * 100                    AS erkennungsrate
FROM score_db
GROUP BY periode, verwendetes_modul
ORDER BY periode, modell;
```

3. Predictability der Messgrößen im Zeitverlauf

Aufbauend auf dem in Kapitel 1.1 dargestellten Forschungsstand (DIQA) wird auch die Vorhersagekraft («Predictability») der einzelnen Messgrößen für den OCR-Erfolg über die Zeit beobachtet. Als Predictability-Wert dient der vom Housekeeping-Prozess (Kapitel 8) je Messgrösse ermittelte Gewichtung- bzw. Feature-Importance-Faktor w_i , ausgewertet über die Zeitreihen-DB (Kapitel 5.2). Dargestellt wird je Messgrösse eine Linie über die aufeinanderfolgenden Housekeeping-Läufe.

```

SELECT
    zeitstempel_letzte_aktualisierung          AS housekeeping_lauf,
    messgroesse_id,
    verarbeitungsmodul,
    gewichtungsfaktor_w                      AS predictability
FROM zeitreihen_db
ORDER BY messgroesse_id, verarbeitungsmodul, housekeeping_lauf;

```

Verliert eine Messgrösse über mehrere Housekeeping-Läufe hinweg spürbar an Gewicht bzw. Vorhersagekraft, ist das ein Hinweis, dass sich die zugrunde liegenden Bedingungen geändert haben (z. B. andere Scanner-/Beleg-Typen, verändertes LLM-Verhalten) und dass der Messgrössen-Katalog (Anhang A) überprüft, angepasst oder um neue Messgrössen ergänzt werden sollte — die Erweiterbarkeit des Katalogs ist hierfür bewusst vorgesehen (Kapitel 2).

Datenbank-Anbindung des Dashboards (Architekturvorschlag)

Kernprinzip: Das Dashboard-Frontend greift nie direkt auf die Datenbanken zu, sondern ausschliesslich über eine schmale Read-Only-Schicht. Die Zugriffskette sieht wie folgt aus:

```

Browser (Administrator)
| HTTPS
▼
Reverse Proxy (Traefik/nginx, TLS-Terminierung)
| OIDC-Login (Keycloak, siehe Kapitel 10)
▼
Dashboard-Backend (Container, Read-Only-Endpunkte)
| interner DB-User «dashboard_reader», nur SELECT
▼
SCORE-DB + Zeitreihen-DB

```

Diese Zwischenschicht bringt drei Vorteile: Erstens verbleiben die Datenbank-Zugangsdaten im Backend-Container (Docker-Secret) und der DB-Benutzer `dashboard_reader` erhält ausschliesslich SELECT-Rechte auf definierte Views — im Einklang mit dem Least-Privilege-Prinzip (Kapitel 10). Zweitens laufen alle Dashboard-Abfragen gegen vordefinierte, aggregierende Views statt gegen freie SQL-Queries, sodass kein Dashboard-Zugriff die Live-Verarbeitung des Hauptprozesses ausbremsen kann. Drittens bleibt die Frontend-Technologie frei austauschbar, ohne dass sich am Datenbankzugriff etwas ändert.

Umsetzung in drei Schritten:

Schritt 1 — Datenbankseite: Für jede der oben definierten Ansichten wird eine dedizierte View angelegt und einem minimal berechtigten Lese-Benutzer zugewiesen:

```

CREATE VIEW v_dash_erkennungsrate_gesamt AS ...; -- Ansicht 1
CREATE VIEW v_dash_modellqualitaet AS ...;      -- Ansicht 2
CREATE VIEW v_dash_predictability AS ...;       -- Ansicht 3 (Zeitreihen-DB)
CREATE VIEW v_dash_bestand AS ...;             -- Bestandskennzahlen

CREATE USER dashboard_reader WITH PASSWORD '...';
GRANT SELECT ON v_dash_erkennungsrate_gesamt,
              v_dash_modellqualitaet,
              v_dash_predictability,
              v_dash_bestand
TO dashboard_reader;                          -- keine weiteren Rechte

```

Da die Zeitreihen-DB nur wöchentlich vom Housekeeping beschrieben wird, empfiehlt sich der Einsatz *materialisierter Views*, die am Ende jedes Housekeeping-Laufs aktualisiert werden (ein zusätzlicher n8n-Schritt: `REFRESH MATERIALIZED VIEW ...`). Jeder Dashboard-Aufruf kostet dann nur Millisekunden, unabhängig von der Grösse der SCORE-DB.

Schritt 2 — Backend: Der bereits vorgesehene Python-Microservice (LightGBM/FAISS, siehe oben) wird um wenige Read-Only-Endpunkte erweitert (z. B. `GET /api/dashboard/erkennungsrate`, `/modellqualitaet`, `/predictability`, `/bestand`), die die Views abfragen und die Ergebnisse als chart-taugliches JSON liefern. Ein zusätzlicher Container ist dafür nicht erforderlich.

Schritt 3 — Frontend: Zwei realistische Optionen:

OPTION	AUFWAND	BEWERTUNG
Grafana (eigener Container)	sehr gering — verbindet sich direkt als <code>dashboard_reader</code> mit den Views; Panels per Konfiguration; OIDC-Anbindung an Keycloak integriert; Schritt 2 (API) entfällt in diesem Fall sogar	Empfohlen für den Start
Eigenes Web-Frontend (z. B. React + Chart.js) gegen die FastAPI-Endpunkte	mittel — volle Gestaltungsfreiheit, entspricht dem Mockup in Anhang B	Sinnvoll, sobald das Dashboard auch Aktionen auslösen soll (z. B. Modul deaktivieren)

Empfehlung: Start mit Grafana auf Basis der Read-Only-Views (schnellster Weg, Authentifizierung via Keycloak ohne Eigenentwicklung). Kommen später interaktive Administrations-Aktionen hinzu, erfolgt die Migration auf das eigene Frontend mit FastAPI-Backend — die Views bleiben dabei unverändert die gemeinsame Datenbasis beider Varianten.

Zielgruppe des Dashboards ist der Administrator von SCORE, nicht das aufrufende System (vgl. Abgrenzung, Kapitel 2.1). Konkrete Visualisierungskomponente, Aktualisierungsfrequenz und Zugriffsschutz des Dashboards sind technisch noch zu spezifizieren (siehe Kapitel 10, offene Punkte); der vorstehende Architekturvorschlag dient als Entscheidungsgrundlage.

8.3 Sicherheitskonzept (Empfehlungen)

Für die Authentisierung und Autorisierung aller beteiligten Komponenten — Docker-Container (n8n-Workflows, Python-Microservices für ML und Redundanzanalyse), Datenbanken sowie das Administrations-Dashboard (Kapitel 8.2) — werden folgende Massnahmen empfohlen. Die verbindliche Festlegung erfolgt gemäss Kapitel 10 (offene Punkte); dieser Abschnitt dient als Entscheidungsgrundlage.

Dienst-zu-Dienst-Kommunikation (Container untereinander)

- Für die Aufrufe zwischen den Containern (n8n → HTTP-Endpunkte des Hauptprozesses, n8n → Python-Microservices) werden einfache, **pro Dienst getrennte API-Keys** bzw. Bearer-Token verwendet.
- Die Schlüssel werden als **Docker-Secrets** bzw. Umgebungsvariablen bereitgestellt — nie im Code oder in Workflow-Definitionen hinterlegt.
- Die interne Kommunikation läuft ausschliesslich über ein **internes Docker-Netzwerk**; interne Dienste erhalten keine öffentliche Port-Freigabe.

Datenbankzugriffe (Least Privilege)

- Jeder Dienst erhält einen **dedizierten, minimal berechtigten DB-Benutzer**: Der Hauptprozess darf in der SCORE-DB lesen und schreiben, aber nicht löschen; nur der Housekeeping-Dienst besitzt Löschrechte (für die Batch-Löschung gemäss Kapitel 8.1); das Dashboard nutzt den reinen Lese-Benutzer `dashboard_reader` (siehe Kapitel 8.2, Datenbank-Anbindung).

Administrations-Dashboard (menschlicher Zugriff)

- Anmeldung über **OpenID Connect / OAuth 2.0** gegen einen leichtgewichtigen, selbst gehosteten Identity Provider (z. B. **Keycloak** oder Authentik als eigener Container).
- **Rollenbasierte Autorisierung** mit mindestens der Rolle «Administrator».
- Optional kann derselbe Identity Provider mittels *Client-Credentials-Flow* mittelfristig auch die Dienst-zu-Dienst-Token ausstellen und damit die API-Keys ablösen.

Externe Erreichbarkeit

- Alle von aussen erreichbaren Endpunkte — Hauptprozess (FA-11), Rückmelde-Endpunkt (FA-19) und Dashboard — werden hinter einem **Reverse Proxy** (z. B. Traefik oder nginx) mit **TLS-Terminierung** gebündelt; nur der Proxy ist öffentlich exponiert.

9. Nicht-funktionale Anforderungen

- **Performance:** Die statische Bildanalyse muss deutlich schneller als ein OCR-/LLM-Aufruf sein (Millisekunden- bis niedriger Sekundenbereich).
- **Erweiterbarkeit:** Neue Messgrößen oder neue Verarbeitungsmodule müssen ohne Strukturänderung der SCORE-DB ergänzbar sein (siehe Kapitel 6.3).
- **Nachvollziehbarkeit:** Zu jeder automatischen Modulwahl müssen der referenzierte historische Datensatz und die berechnete Distanz gespeichert werden.
- **Kostentransparenz:** Tokenverbrauch und Kosten müssen je Verarbeitungsfall und aggregiert (z. B. je Modul, je Zeitraum) auswertbar sein.
- **Nachträgliche Aktualisierbarkeit:** Status-Updates aus Folgesystemen müssen auch Tage oder Wochen nach Erstverarbeitung noch dem richtigen Datensatz zugeordnet werden können.
- **Konfigurierbarkeit:** Kosten- und Gewichtungsfaktoren müssen ohne Deployment, z. B. über die Zeitreihen-DB, änderbar sein.
- **Verschlüsselung der externen Kommunikation:** Sämtliche externe Kommunikation ist zu verschlüsseln. Alle von aussen erreichbaren Schnittstellen (Aufruf des Hauptprozesses FA-11, Rückmelde-Endpunkt FA-19, Administrations-Dashboard) werden ausschliesslich über TLS-verschlüsseltes HTTPS nach aktuellem Standard betrieben (TLS 1.3 bevorzugt, TLS 1.2 als Fallback; siehe Kapitel 13.1). Unverschlüsselte Verbindungen werden abgewiesen bzw. auf HTTPS umgeleitet.
- **Informationssicherheit (hohe Anforderungen):** Das Gesamtsystem muss sehr hohen Sicherheitsanforderungen genügen und ist an den Vorgaben des BSI IT-Grundschutzes sowie der ISO/IEC 27001 auszurichten. Dies umfasst unter anderem ein Informationssicherheits-Managementsystem (ISMS) mit Risikoanalyse, die konsequente Umsetzung des Least-Privilege-Prinzips für Dienste und Datenbankzugriffe (Kapitel 8.3), Verschlüsselung der Daten bei Übertragung und — soweit erforderlich — im ruhenden Zustand, nachvollziehbare Protokollierung sicherheitsrelevanter Ereignisse sowie ein geregeltes Patch- und Schwachstellenmanagement für alle beteiligten Container. Die konkrete Zertifizierungs- bzw. Nachweisstrategie ist festzulegen (siehe Kapitel 10, offene Punkte).

10. Offene Punkte / Annahmen

- Schwellenwerte, ab wann ein Ergebnis als «erfolgreich» vs. «manuelle Nachbearbeitung nötig» gilt, sind zu spezifizieren (z. B. OCR-Konfidenz, Validierungsregeln).
- Genaues Format des Identifikators, den DATEV oder andere Folgesysteme beim Rückmelde-Endpunkt (11.6) übermitteln, ist mit dem jeweiligen Folgesystem abzustimmen.
- Format der Angabe von `max_kosten` (Währung, Nachkommastellen) sowie Rundungsregeln sind festzulegen.
- Vorgehen bei weniger als drei verfügbaren Vergleichsdatensätzen in der SCORE-DB (Kaltstart-Fall) ist zu definieren; die Eskalations-View aus Kapitel 6.2 sowie der Onboarding-Mechanismus aus Kapitel 6.3 liefern hierfür einen Lösungsansatz, ersetzen aber nicht die abschliessende fachliche Festlegung.
- Endgültige Festlegung des ML-Verfahrens des Housekeeping-Prozesses (empfohlen: LightGBM, siehe Kapitel 8) sowie Mindestdatenmenge je Trainingslauf sind final zu bestätigen.
- Vorgehen bei widersprüchlichen oder stark schwankenden Gewichtsänderungen zwischen zwei Housekeeping-Läufen (Glättung/Dämpfung) ist zu klären.
- Zeitpunkt und Kriterium für die Entfernung bzw. Markierung der in Kapitel 6.3 beschriebenen Seed-Datensätze neu onboardeter Module sind festzulegen.
- Konkrete Visualisierungstechnologie (z. B. BI-Tool, eigenes Web-Frontend) sowie Aktualisierungsfrequenz des Administrations-Dashboards (Kapitel 8.2) sind festzulegen; Zugriffsschutz siehe Sicherheitskonzept in Kapitel 8.3.
- Schwellenwerte, ab denen ein Rückgang der Predictability einer Messgrösse (Kapitel 8.2) als kritisch gilt und eine Überarbeitung des Messgrössen-Katalogs (Anhang A) auslöst, sind zu definieren.
- Empfohlener Timeout-Wert für den synchronen Aufruf des Hauptprozesses (Kapitel 11.1) ist festzulegen, abgestimmt auf typische Werte von `max_kosten` und die maximal zu erwartende Anzahl an Eskalationsversuchen.
- Wahl eines geeigneten Authentisierungs- und Autorisierungsverfahrens für alle beteiligten Docker-Container (n8n-Workflows, Python-Microservices, Datenbanken) sowie für das Administrations-Dashboard ist verbindlich zu treffen; die Empfehlungen aus Kapitel 8.3 (API-Keys/Docker-Secrets für Dienst-zu-Dienst, Least-Privilege-DB-Benutzer, OIDC/Keycloak für das Dashboard, Reverse Proxy mit TLS) dienen als Entscheidungsgrundlage.
- Potenzieller Eigenwert der SCORE-Daten bei starker Skalierung: Zu prüfen ist, ob die Erkenntnisse zur Qualität der OCR-/LLM-Modelle und zur Gewichtung der Messgrössen (analog zum Dashboard) regelmässig in einem Blog veröffentlicht werden. Eine kommerzielle Verwertung erscheint fraglich; ein Interesse könnte eher im wissenschaftlichen Umfeld bestehen.
- Backup-Strategie für die SCORE-DB: Ist eine dedizierte Sicherungsstrategie erforderlich, und wenn ja, in welcher Form (Frequenz, Aufbewahrung, Wiederherstellungstest)?
- Muss die SCORE-DB als echter Datenbank-Cluster mit verteilten Transaktionen konfiguriert werden (Hochverfügbarkeit, Konsistenz bei parallelem Haupt- und Housekeeping-Zugriff)?
- Zusammenspiel DuckDB / PostgreSQL: Zu klären ist, ob DuckDB für die schnelle analytische Verarbeitung genutzt wird und die Daten bereits beim Schreiben zusätzlich an einen PostgreSQL-Cluster für die längerfristige Persistenz übergeben werden (siehe Kapitel 13.2).

- Auswahl der produktiv angebotenen Ähnlichkeitsmasse (Kapitel 7.1): Zu entscheiden ist, ob neben der gewichteten euklidischen Distanz weitere Masse (Manhattan, Cosinus, Tschebyschow) — ggf. als wählbarer Parameter analog zu `strategie` — angeboten werden und ob die VSS-Extension (HNSW-Index) für die beschleunigte Suche eingesetzt wird.
- Sicherheits-Nachweisstrategie: Festzulegen ist, in welchem Umfang das Gesamtsystem nach BSI IT-Grundschutz und ISO/IEC 27001 (Kapitel 9) abgesichert und nachgewiesen wird — insbesondere, ob eine formale ISO-27001-Zertifizierung angestrebt wird, welcher Geltungsbereich (Scope) gilt und ob zusätzliche Nachweise (z. B. BSI C5) erforderlich sind. Damit verbunden ist die Frage, ob Daten auch im ruhenden Zustand (Encryption at Rest) verschlüsselt werden müssen.

11. Schnittstellenspezifikation (HTTPS-API)

11.1 Aufruf des Hauptprozesses

Der Hauptverarbeitungsprozess wird über einen HTTPS-Request (POST, TLS-verschlüsselt gemäss Kapitel 13) gestartet. Übergeben werden:

- Bildinhalt des Belegs (z. B. Base64-kodiert oder als `multipart/form-data`).
- `max_kosten`: maximal für die OCR-Verarbeitung dieses Belegs zulässiger Geldbetrag.
- `strategie`: Steuert die Ermittlung des nächsten Verarbeitungsmoduls über die Eskalations-View (Kapitel 6.2). Zulässige Werte: `cost` (günstigstes Modul wird bevorzugt) oder `quality` (Modul mit der höchsten Erkennungsrate der letzten 100 Tage wird bevorzugt).
- `plichtfelder`: JSON-Struktur mit den Feldern, die aus dem Beleg zwingend erkannt werden müssen.

Synchrones Aufrufmodell: Der Aufruf ist synchron — der Aufrufer wartet auf die Antwort des Hauptprozesses und erhält das vollständige Ergebnis (Kapitel 11.4) in der HTTP-Response desselben Requests; es gibt kein Callback- oder Polling-Verfahren. Da der Hauptprozess bei Eskalation mehrere Modelle nacheinander aufrufen kann (Kapitel 3, Schritt 6), muss der Aufrufer ein entsprechend grosszügiges Timeout vorsehen; die Obergrenze der Verarbeitungsdauer ist faktisch durch `max_kosten` begrenzt, da die Eskalation spätestens bei Erreichen der Kostenobergrenze abbricht.

11.2 Beispiel Request-Body

```
{
  "bild": "<Base64-kodierter Bildinhalt>",
  "max_kosten": 0.05,
  "strategie": "cost",
  "pflichtfelder": ["belegdatum", "betrag", "lieferant", "belegnummer"]
}
```

11.3 Kostenobergrenze und Fallback

Kann anhand der Fuzzy-Suche kein Verarbeitungsverfahren ermittelt werden, dessen erwartete Kosten unterhalb von `max_kosten` liegen, wird mindestens das lokale Low-Cost-OCR-Verfahren ausgeführt — auch wenn dessen Ergebnisqualität für den konkreten Beleg unter Umständen nicht ausreicht. In diesem Fall wird der Status entsprechend als «manuelle Nachbearbeitung erforderlich» markiert.

11.4 Beispiel Response-Body

```
{
  "erkannte_felder": {
    "belegdatum": "2026-07-10",
    "betrag": 129.90,
    "lieferant": "Muster GmbH",
    "belegnummer": "RE-2026-4471"
  },
  "verarbeitungsinfo": {
    "messgroessen_vektor": [ /* 60 normierte Werte 0-100 */ ],
    "referenz_datensatz_pk": "8841",
    "fuzzy_distanz": 0.12,
    "verwendetes_modul": "LLM klein",
    "token_anzahl": 1840,
    "kosten": 0.031,
    "status": "erfolgreich"
  }
}
```

11.5 HTTP-Statuscodes

Der Hauptprozess liefert die üblichen HTTP-Returncodes, u. a.:

- **200 OK** — Verarbeitung erfolgreich durchgeführt (Status im Response-Body kann dennoch «manuelle Nachbearbeitung erforderlich» sein).
- **400 Bad Request** — Eingabedaten fehlerhaft (z. B. Bildinhalt fehlt, ungültige JSON-Struktur bei Pflichtfeldern).
- **422 Unprocessable Entity** — Bilddatei nicht lesbar / kein gültiges Bildformat.
- **500 Internal Server Error** — unerwarteter Fehler im Verarbeitungsprozess.
- **503 Service Unavailable** — nachgelagertes OCR-/LLM-Modul aktuell nicht erreichbar.

11.6 Rückmelde-Endpunkt (Status-Update aus Folgesystemen)

Die Rückmeldung einer später erfolgten manuellen Nachbearbeitung (z. B. in DATEV) erfolgt über einen separaten, TLS-verschlüsselten HTTPS-Request-Endpunkt. Dieser übermittelt einen Identifikator des ursprünglichen Belegs/Datensatzes und aktualisiert den entsprechenden Datensatz in der SCORE-DB — unabhängig davon, wie viel Zeit seit der ursprünglichen Verarbeitung vergangen ist.

Die Maske zur Prüfung/Ergänzung der Daten sowie die fachliche Entscheidung, den Fall als «nicht erfolgreiches OCR» zu kennzeichnen, liegen beim aufrufenden System (siehe Abgrenzung, Kapitel 2.1). Dieser Endpunkt dient ausschliesslich der technischen Rückmeldung des dort ermittelten Ergebnisses an SCORE.

Beispiel Request-Body:

```
{
  "datensatz_pk": "8841",
  "status": "manuelle Nachbearbeitung erfolgt",
  "referenz_folgesystem": "DATEV-Beleg-2026-00931",
  "zeitstempel_nachbearbeitung": "2026-07-14T09:32:00Z"
}
```

Der Endpunkt liefert ebenfalls die üblichen HTTP-Returncodes, u. a. 200 OK bei erfolgreichem Update, 404 Not Found falls der referenzierte Datensatz-PK nicht existiert, sowie 400 Bad Request bei fehlerhafter Anfrage.

12. Alternative Ähnlichkeitsverfahren (Ausblick)

Neben der gewichteten quadrierten euklidischen Distanz (Kapitel 7) sind für die Ähnlichkeitssuche auf numerischen Messgrößen-Vektoren weitere Verfahren denkbar, die im Rahmen künftiger Weiterentwicklung geprüft werden können:

- **Manhattan-Distanz (L1-Norm):** Summe der absoluten statt quadrierten Differenzen — reagiert weniger empfindlich auf einzelne, stark abweichende Messgrößen (Ausreisser).
- **Mahalanobis-Distanz:** berücksichtigt zusätzlich Streuung und Korrelation zwischen den Messgrößen und normiert dadurch automatisch unterschiedliche Wertebereiche, statt dies über feste Gewichte zu tun.
- **Kosinus-Ähnlichkeit:** bewertet den Winkel zwischen zwei Messgrößen-Vektoren statt ihrer absoluten Distanz — geeignet, wenn das Verhältnis der Messgrößen zueinander wichtiger ist als deren absolute Höhe.
- **Klassische Fuzzy-Logik (Mamdani-Ansatz):** jede Messgröße wird über Zugehörigkeitsfunktionen (z. B. «niedrig/mittel/hoch») in Fuzzy-Werte überführt und über Wenn-Dann-Regeln zu einem Gesamt-Ähnlichkeitsgrad kombiniert — bildet Expertenwissen explizit ab, statt rein statistisch zu wirken.
- **Gewichtetes k-Nearest-Neighbour-Verfahren mit Kernel-Gewichtung:** statt der Auswahl des einzelnen nächsten Nachbarn fließen mehr Nachbarn ein, deren Einfluss mit wachsender Distanz über eine Kernel-Funktion (z. B. Gauss-Kernel) abnimmt.
- **Fuzzy-C-Means-Clustering:** die Datensätze der SCORE-DB werden vorab in Cluster ähnlicher Belegtypen gruppiert; die aktuelle Datei wird dem/den ähnlichsten Cluster(n) zugeordnet, statt jedes Mal alle Einzeldatensätze zu vergleichen.
- **Baumbasierte Ähnlichkeit (z. B. Random-Forest-Proximity):** misst, wie oft zwei Datensätze in einem trainierten Entscheidungsbaum-Ensemble im selben Blatt landen — kann nichtlineare Zusammenhänge zwischen Messgrößen erfassen, die eine einfache Distanzformel nicht abbildet.

Diese Verfahren sind grundsätzlich mit dem in Kapitel 8 beschriebenen Housekeeping-Prozess kombinierbar, z. B. indem das ML-Modell dort direkt Mahalanobis-Kovarianzen oder Cluster-Zentren statt einfacher Gewichtungsfaktoren ableitet.

13. Deployment

13.1 TLS-Verschlüsselung aller HTTPS-Schnittstellen

Sämtliche HTTP-Schnittstellen von SCORE — der Aufruf des Hauptprozesses (FA-11), der Rückmelde-Endpunkt (FA-19) und der Zugriff auf das Administrations-Dashboard (Kapitel 8.2) — werden ausschliesslich über HTTPS mit TLS-Verschlüsselung nach aktuellem Standard betrieben. Konkret gilt:

- **TLS 1.3** (RFC 8446) als bevorzugtes Protokoll — der aktuelle Stand der Technik mit reduziertem Handshake, ausschliesslich AEAD-Cipher-Suiten und verpflichtender Forward Secrecy.
- **TLS 1.2** (RFC 5246) mit modernen Cipher-Suiten (ECDHE-Schlüsselaustausch, AES-GCM oder ChaCha20-Poly1305) nur als Fallback für Clients, die TLS 1.3 noch nicht unterstützen.
- **TLS 1.0 und TLS 1.1** werden vollständig deaktiviert; sie sind gemäss RFC 8996 formal veraltet und von allen gängigen Browsern und Compliance-Vorgaben ausgeschlossen. Ebenso werden SSL 2.0/3.0 sowie schwache Cipher (RC4, CBC, SHA-1, MD5, RSA-Schlüsselaustausch ohne Forward Secrecy) abgeschaltet.

Die TLS-Terminierung erfolgt zentral am Reverse Proxy (Kapitel 8.3, Externe Erreichbarkeit); nur dieser ist öffentlich erreichbar. Die interne Dienst-zu-Dienst-Kommunikation läuft im internen Container-Netzwerk. Als Zielbild ist zu berücksichtigen, dass TLS 1.3 die einzige Protokollarchitektur ist, die den kommenden Übergang zu Post-Quantum-Kryptografie (hybride Schlüsselverfahren) unterstützt.

13.2 Docker-Container

SCORE wird als Verbund von Docker-Containern betrieben. Die folgende Liste nennt die vorgesehenen Container mit ihrer Aufgabe und einem Verweis auf das jeweilige Werkzeug:

CONTAINER	AUFGABE IM SCORE-SYSTEM	WEBSITE
n8n	Orchestrierung des Haupt-, Rückmelde- und Housekeeping-Workflows	n8n.io
Tesseract OCR	Lokales Low-Cost-OCR-Modul (On-Site). PDF-Dateien werden im Hauptprozess grundsätzlich an diesen lokalen OCR-Container übergeben — andere Verfahren (LLM) sind für PDF nicht sinnvoll. Dieser Container ist in der Skillsprinter-Umgebung deployed.	github.com/tesseract-ocr/tesseract
Python-Microservice (ML)	Wöchentliche Feature-Importance-/Predictability-Berechnung mit LightGBM (Kapitel 8) sowie Redundanzanalyse mit FAISS (Kapitel 8.1)	lightgbm.readthedocs.io · FAISS
DuckDB	Extrem schnelle, analytische In-Memory-Datenbank mit nativer Unterstützung für den Datentyp JSON; dient der schnellen analytischen Verarbeitung (u. a. Distanzberechnung, Dashboard-Aggregationen)	duckdb.org
PostgreSQL-Cluster	Persistente, hochverfügbare Speicherung der SCORE-DB und der Zeitreihen-DB für die längerfristige Persistenz (verteilter Betrieb, siehe offene Punkte in Kapitel 10)	postgresql.org
Keycloak	Identity Provider für OIDC/OAuth 2.0-Anmeldung am Dashboard (Kapitel 8.3)	keycloak.org
Reverse Proxy (Traefik)	TLS-Terminierung und Bündelung der extern erreichbaren Endpunkte (Kapitel 8.3, 13.1)	traefik.io
Grafana (optional)	Visualisierung des Administrations-Dashboards auf Basis der Read-Only-Views (Kapitel 8.2)	grafana.com

Zum Zusammenspiel der beiden Datenbanken siehe die offenen Punkte in Kapitel 10: Zu klären ist, ob DuckDB für die schnelle analytische Verarbeitung genutzt wird und die Daten bereits beim Schreiben zusätzlich an einen PostgreSQL-Cluster für die längerfristige Persistenz übergeben werden.

13.3 Betriebsumgebung: Failover- und Skalierungscluster

Für einen hochverfügbaren, skalierbaren Betrieb der Container empfiehlt sich eine Managed-Kubernetes-Umgebung mit Auto-Scaling und geo-redundanter Verteilung. Da die verarbeiteten Belegdaten personenbezogen sein können, sind Anbieter mit Rechenzentren in der EU und Governance nach EU-Recht (kein Zugriff nach US-CLOUD-Act) zu bevorzugen.

IONOS Cloud Managed Kubernetes (Deutschland) bietet einen solchen Betrieb: geo-redundant verteilter Control Plane (mehrere Replikas an verschiedenen Standorten innerhalb Deutschlands), Node-Pools über mehrere Rechenzentren hinweg für Geo-Redundanz, Auto-Scaling nach Bedarf sowie integrierte persistente Volumes (CSI). Website: cloud.ionos.com/managed/kubernetes.

Weitere EU-Anbieter mit vergleichbarem Managed-Kubernetes-Angebot und EU-Rechenzentren:

ANBIETER	SITZ / RECHENZENTREN (EU)	WEBSITE
OVHcloud	Frankreich; 30+ Rechenzentren europaweit	ovhcloud.com
Scaleway	Frankreich; Rechenzentren Paris, Amsterdam, Warschau	scaleway.com
STACKIT (Schwarz Gruppe)	Deutschland und Österreich	stackit.de
T Cloud Public / Open Telekom Cloud (Deutsche Telekom)	Deutschland und weitere EU-Standorte	telekom.de
Exoscale	Schweiz/EU; mehrere europäische Zonen	exoscale.com
UpCloud	Finnland; mehrere EU-Rechenzentren	upcloud.com

Hetzner (Deutschland, hetzner.com) bietet zwar kein Managed Kubernetes, ist jedoch als kostengünstige Basis für selbstverwaltete Cluster (z. B. k3s) verbreitet und daher als preisgünstige Alternative erwähnenswert.

14. Testscenarien

Verfügbare Testdaten: Für den Massentest (13.2) sowie für den Aufbau eines initialen, breiten Datenbestands für die Ähnlichkeitssuche (Kapitel 7) und den Housekeeping-Prozess (Kapitel 8) steht ein öffentlich verfügbarer Datensatz zur Verfügung: Der Hugging-Face-Datensatz `mychen76/invoices-and-receipts_ocr_v1` umfasst mehrere tausend Belegdateien (Rechnungen und Quittungen) mit bereits erkanntem Referenztext (Ground Truth). Ein Hugging-Face-Read-Token für den Zugriff auf den Datensatz ist vorhanden.

14.1 Integrationstest (Einzeldatei-Workflow)

Ziel: Verifikation, dass der Gesamtworkflow von der Dateiübergabe bis zur Rückgabe des OCR-Ergebnisses und dem Anlegen des zugehörigen Datenbank-Datensatzes technisch korrekt funktioniert.

- Ein n8n-Workflow liest eine einzelne Testdatei ein.
- Der Workflow startet über den HTTP-Request-Endpoint (FA-11) den Hauptverarbeitungsprozess mit dieser Testdatei.
- Es wird geprüft, dass eine valide Response (erkannte Felder gemäss Pflichtfeldern sowie Verarbeitungsinfo mit Messgrößen-Vektor, Referenzdatensatz und Kosten) sowie der erwartete HTTP-Statuscode zurückgeliefert werden.
- Es wird geprüft, dass in der SCORE-DB ein neuer, konsistenter Datensatz mit PK, Zeitstempel, Messgrößen-Vektor, verwendetem Modul und Status angelegt wurde.

Abnahmekriterium: Die Response enthält alle geforderten Pflichtfelder und einen plausiblen Status; der zugehörige Datenbank-Datensatz ist vorhanden und stimmt mit der Response überein.

14.2 Massentest (Volumen- und Regressionstest)

Ziel: Prüfung der Verarbeitungsqualität und -stabilität über eine grosse, repräsentative Menge realer Belegtypen sowie Aufbau eines initialen Datenbestands für die Fuzzy-Suche (Kapitel 7) und den Housekeeping-Prozess (Kapitel 8).

Voraussetzung: Es liegt ein Korpus von mehreren zehntausend Testdateien vor (siehe Hinweis zu Beginn von Kapitel 14), zu denen jeweils die korrekt erkannten Felder (Referenzdaten / Ground Truth) bereits bekannt sind.

1. Ein n8n-Workflow iteriert über den gesamten Testdatei-Korpus (mehrere zehntausend Dateien).
2. Für jede Testdatei wird der Hauptverarbeitungsprozess über die HTTP-Schnittstelle gestartet.
3. Die vom Hauptprozess erkannten Felder werden automatisiert mit den hinterlegten Referenzdaten der jeweiligen Testdatei verglichen.
4. Bei einer Abweichung wird der Rückmelde-Prozess (FA-19, Kapitel 11.6) zur Dokumentation einer erforderlichen Nachbearbeitung aufgerufen und der zugehörige Datensatz entsprechend als «manuelle Nachbearbeitung erforderlich» markiert.
5. Bei Übereinstimmung von erkannten Feldern und Referenzdaten bleibt der Datensatz-Status «erfolgreich».

Ergebnis: Es entsteht eine dem Testkorpus entsprechend grosse Anzahl an Datensätzen in der SCORE-DB (Grössenordnung Zehntausende), die als Vergleichs- und Trainingsbasis für die Fuzzy-Suche sowie den Housekeeping-Prozess dient.

Zusätzlich auszuwertende Kennzahlen des Massentests:

- Trefferquote (Anteil «erfolgreich» vs. «manuelle Nachbearbeitung erforderlich») je Verarbeitungsmodul und je Belegtyp.
 - Verteilung der automatisch gewählten Verarbeitungsmodule über den gesamten Testkorpus.
 - Gesamtkosten sowie Kosten je Verfahren über den gesamten Testlauf.
 - Durchsatz/Performance (Anzahl verarbeiteter Dateien pro Zeiteinheit) zur Abschätzung der Skalierbarkeit.
-

Anhang A: Messgrößen-Katalog (60 Vor-OCR-Metriken)

Der in FA-1 referenzierte Messgrößen-Vektor besteht aus den folgenden 60 Bildqualitäts- und Bildeigenschaften-Metriken, die vor jedem OCR-Aufruf rein statisch aus der Bilddatei ermittelt werden (Details siehe separates Excel «OCR_Vor_Metriken.xlsx»). Jede Metrik wird auf einen Wert zwischen 0 und 100 normiert.

#	MESSGRÖSSE	NORMIERUNG (0-100)	AUFWAND	LIBRARY	N8N
1	Bildauflösung (PPI rel. Zeichenhöhe)	$\min(100, 100 \times \text{dpi} / 300)$	gering	sharp	Ja (Mo
2	Kontrast (RMS-Kontrast)	$\min(100, 100 \times \text{rms} / 64)$	gering-mittel	jimp	Ja (Mo
3	Schärfe (Laplace-Varianz)	$\min(100, 100 \times \text{var} / 500)$	mittel	opencv4nodejs	Ja (Mo
4	Belichtung (Histogramm-Clipping)	$100 - \text{Clip-Anteil} \times 100$	gering	jimp	Ja (Mo
5	Bildrauschen (Std-Abw. homogene Flächen)	$100 - \min(100, \text{std} \times 2)$	mittel	jimp	Ja (Mo
6	Schräglage/Skew-Winkel (Hough)	$100 - \min(100, \text{Winkel} \times 10)$	hoch	opencv4nodejs	Ja (Mo
7	Otsu-Trennschärfe (Inter-Klassen-Varianz)	$\min(100, 100 \times \text{var} / \text{var_max})$	mittel	jimp	Ja (Mo
8	Kantendichte (Sobel-Gradient)	$\min(100, 100 \times \text{Kantenpixel} / \text{Gesamtpixel} \times k)$	mittel-hoch	opencv4nodejs	Ja (Mo
9	Tinten-Flächenanteil (Ink Coverage)	$100 \times \min(1, \text{Coverage} / \text{Referenz})$	gering-mittel	jimp	Ja (Mo
10	Kompressionsartefakte (Blockiness 8x8)	$100 - \min(100, \text{Blockiness} \times f)$	mittel	jimp	Ja (Mo
11	Farbtiefe/Kanalanzahl	Graustufe=100, RGB ohne Vorverarb.=60	trivial	sharp	Ja (Mo
12	Byte pro Pixel (Kompressionsrate)	$\min(100, 100 \times \text{bpp} / \text{bpp_ref})$	trivial	keine	Ja (nat
13	Seitenverhältnis-Plausibilität	100 im Toleranzband, sonst Abstufung	trivial	keine	Ja (nat
14	Bildgrösse absolut (Pixelzahl)	$\min(100, 100 \times \text{px} / \text{px_ref})$	trivial	keine	Ja (nat
15	Farbstich/Cast (Kanalabweichung)	$100 - \min(100, \Delta \times f)$	gering	sharp	Ja (Mo
16	Sättigung (Ø-Saturation)	$100 - \min(100, \text{sat} \times f)$	gering	jimp	Ja (Mo
17	Dynamikumfang (Histogramm-Spannweite)	$\min(100, 100 \times \text{range} / 255)$	gering	jimp	Ja (Mo
18	Bild-Entropie	$\min(100, 100 \times H / H_max)$	mittel	jimp (eigen)	Ja (Mo
19	Histogramm-Peakanzahl (Bimodalität)	100 bei 2 klaren Peaks, sonst Abstufung	mittel	jimp	Ja (Mo
20	Weissraum-Anteil	$100 \times \min(1, \text{ws} / \text{ref})$	gering	jimp	Ja (Mo
21	Vignettierung/Beleuchtungsgradient	$100 - \min(100, \text{grad} \times f)$	mittel	jimp	Ja (Mo
22	Schattenanteil an Rändern	$100 - \min(100, \text{Schatten} \times f)$	mittel	jimp	Ja (Mo
23	Glanzlicht/Specular-Highlight-Fläche	$100 - \min(100, \text{Glanz} \times f)$	mittel	opencv4nodejs	Ja (Mo
24	Bildrahmen/Randbeschnitt vorhanden	100 wenn sauber, sonst Abzug	gering	jimp	Ja (Mo
25	Seitenwölbung (Buchscan-	$100 - \min(100, \text{Krümmung} \times f)$	hoch	opencv4nodejs	Ja (Mo

	Krümmung)				aufwär
26	Perspektivische Verzerrung	$100 - \min(100, \text{Verzerrung} \times f)$	hoch	opencv4nodejs	Ja (Mo
27	Bildrotation (0/90/180/270 grob)	100 wenn 0°, sonst Abstufung	mittel	jimp / Projektionsprofil	Ja (Mo
28	Moiré-Muster-Stärke (FFT-Peak)	$100 - \min(100, \text{Peak} \times f)$	hoch	fft-js	Ja (Mo komple
29	Farbquantisierungs-Stufen	$\min(100, 100 \times \text{Stufen} / 256)$	gering	jimp	Ja (Mo
30	Salt-and-Pepper-Rauschanteil	$100 - \min(100, \text{Anteil} \times f)$	mittel	jimp	Ja (Mo
31	Bewegungsunschärfe (Richtungs-Gradient)	$100 - \min(100, \text{Motion} \times f)$	hoch	opencv4nodejs	Ja (Mo
32	Gaussian-Blur-Schätzwert	$100 - \min(100, \text{Sigma} \times f)$	mittel	opencv4nodejs	Ja (Mo
33	Farbkanal-Rauschverhältnis (SNR)	$\min(100, 100 \times \text{snr} / \text{snr_ref})$	mittel	jimp	Ja (Mo
34	Chromatische Aberration (Kantenversatz RGB)	$100 - \min(100, \text{Versatz} \times f)$	hoch	opencv4nodejs	Ja (Mo
35	Bit-Tiefe (8/16/32 Bit)	8bit=100, höher=Abstufung je nach Nutzung	trivial	sharp	Ja (Mo
36	Dateiformat-Kompressionstyp (verlustfrei/-behaftet)	verlustfrei=100, JPEG=Abstufung	trivial	sharp / file-type	Ja (Mo
37	JPEG-Qualitätsschätzung (DCT-Analyse)	$\min(100, q_geschätzt)$	hoch	jpeg-js	Ja (Mo komple
38	Lokale Kontrastvarianz (Fensterweise)	$\min(100, 100 \times \text{var} / \text{var_ref})$	mittel- hoch	jimp	Ja (Mo
39	Texturkomplexität (GLCM-Homogenität)	$\min(100, 100 \times \text{hom} / \text{hom_ref})$	hoch	opencv4nodejs	Ja (Mo
40	Symmetrie/Spiegelabweichung (Formulare)	$100 - \min(100, \text{Abw} \times f)$	mittel	jimp	Ja (Mo
41	Hintergrundfarb-Uniformität	$100 - \min(100, \text{std_bg} \times f)$	mittel	jimp	Ja (Mo
42	Staub-/Kratzer-Erkennung (isolierte Ausreisser)	$100 - \min(100, \text{Anzahl} \times f)$	mittel- hoch	opencv4nodejs	Ja (Mo
43	Wasserzeichen- /Stempelüberlagerung	$100 - \text{Overlap} \times 100$	hoch	opencv4nodejs	Ja (Mo aufwär
44	Tabellen-/Linienstruktur (Hough h/v)	100 bei erwarteter Struktur	hoch	opencv4nodejs	Ja (Mo
45	Spaltenanzahl-Schätzung (vertikale Lücken)	100 bei plausibler Spaltenzahl	mittel	jimp (Projektion)	Ja (Mo
46	Textzeilen-Anzahl-Schätzung (horiz. Projektion)	$100 - \text{n_ist} - \text{n_soll} \times f$	mittel	jimp (eigen)	Ja (Mo
47	Zeilenabstand-Konsistenz (Projektionsprofil-Peaks)	$100 - \min(100, \text{var} \times f)$	mittel	jimp (eigen)	Ja (Mo
48	Connected-Components-Anzahl (Layout-Grobmass)	$100 - \min(100, \text{cc_ist} - \text{cc_soll} \times f)$	hoch	opencv4nodejs	Ja (Mo
49	Verhältnis Vorder- /Hintergrundfläche	$100 \times \min(1, \text{ratio} / \text{ref})$	gering- mittel	jimp	Ja (Mo
50	Bildkanten-Beschnitt-Verlust (Randobjekte abgeschnitten)	$100 - \min(100, \text{Anteil} \times f)$	mittel	jimp	Ja (Mo
51	Zeichenhöhe in Pixel (x-Height)	$\min(100, 100 \times \text{px_höhe} / 25)$	mittel	jimp	Ja (Mo

				(Projektionsprofil)	
52	Text-vs-Nichttext-Klassifikation (Textflächenanteil)	100×Textflächenanteil/erwartet	mittel	opencv4nodejs	Ja (Mo
53	Handschrift- vs. Druckschrift-Indikator	Druck=100, Handschrift=20-40	hoch	opencv4nodejs	Ja (Mo aufwär
54	Lokale Schärfe-Varianz (Fokus-Ungleichmässigkeit)	100−min(100,std_lokal×f)	hoch	opencv4nodejs	Ja (Mo
55	Glanzlicht-/Reflexionsflächen (Glare)	100−min(100,Glare-Fläche×f)	mittel	opencv4nodejs	Ja (Mo
56	Durchscheinen der Rückseite (Show-through)	100−min(100,Show-through-Anteil×f)	hoch	opencv4nodejs	Ja (Mo aufwär
57	Strichstärke/Stroke-Width-Konsistenz	100−min(100,var(stroke_width)×f)	hoch	opencv4nodejs	Ja (Mo aufwär
58	Upsampling-/Interpolations-Erkennung	100 wenn nativ, Abzug bei Interpolation	mittel	jimp	Ja (Mo
59	Barcode-/Logo-/Stempel-Overlay auf Textbereich	100−Overlap-mit-Textzone×100	hoch	opencv4nodejs	Ja (Mo aufwär
60	Mehrfach-Kompressionsgenerationen (JPEG-Ghosting)	100−min(100,Ghosting-Score×f)	hoch	jpeg-js	Ja (Mo komple

Die Metriken #1-50 sind reine, allgemeine Bildqualitätschecks; die Metriken #51-60 sind vertiefende, OCR-spezifische Zusatzmetriken (z. B. Handschrift-Erkennung, Show-through, Stroke-Width-Konsistenz), die zusätzliche Vorhersagekraft für den OCR-Erfolg liefern.

A.1 Praktikable Teilmenge (ohne hohen Aufwand / komplexe Module)

Die 60 Messgrössen der obigen Tabelle stellen das grundsätzlich Mögliche dar. Für eine schlanke, produktionsnahe Implementierung (z. B. n8n Cloud oder einfache Node-Umgebungen ohne native Build-Tools) werden nachfolgend alle Messgrössen entfernt, die (a) einen hohen Laufzeitaufwand («hoch» oder «mittel-hoch») verursachen oder (b) ein komplexes, installationsaufwändiges Softwaremodul (opencv4nodejs, fft-js, jpeg-js) benötigen.

Ergebnis: 35 von 60 Messgrössen verbleiben in der praktikablen Teilmenge; 25 werden ausgeschlossen.

Beibehalten (praktikable Teilmenge)

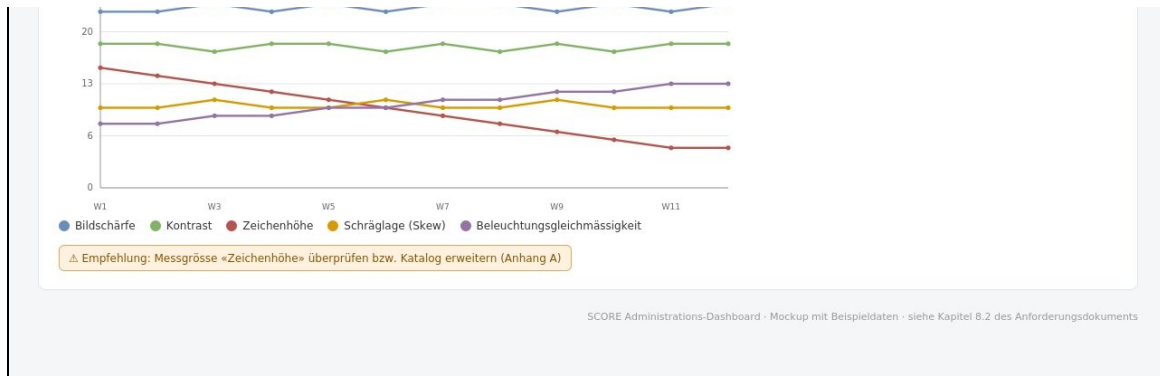
#	MESSGRÖSSE	AUFWAND	LIBRARY	N8N
1	Bildauflösung (PPI rel. Zeichenhöhe)	gering	sharp	Ja (Modul)
2	Kontrast (RMS-Kontrast)	gering-mittel	jimp	Ja (Modul)
4	Belichtung (Histogramm-Clipping)	gering	jimp	Ja (Modul)
5	Bildrauschen (Std-Abw. homogene Flächen)	mittel	jimp	Ja (Modul)
7	Otsu-Trennschärfe (Inter-Klassen-Varianz)	mittel	jimp	Ja (Modul)
9	Tinten-Flächenanteil (Ink Coverage)	gering-mittel	jimp	Ja (Modul)
10	Kompressionsartefakte (Blockiness 8×8)	mittel	jimp	Ja (Modul)
11	Farbtiefe/Kanalanzahl	trivial	sharp	Ja (Modul)
12	Byte pro Pixel (Kompressionsrate)	trivial	keine	Ja (nativ)
13	Seitenverhältnis-Plausibilität	trivial	keine	Ja (nativ)

14	Bildgrösse absolut (Pixelzahl)	trivial	keine	Ja (nativ)
15	Farbstich/Cast (Kanalabweichung)	gering	sharp	Ja (Modul)
16	Sättigung (Ø-Saturation)	gering	jimp	Ja (Modul)
17	Dynamikumfang (Histogramm-Spannweite)	gering	jimp	Ja (Modul)
18	Bild-Entropie	mittel	jimp (eigen)	Ja (Modul)
19	Histogramm-Peakanzahl (Bimodalität)	mittel	jimp	Ja (Modul)
20	Weissraum-Anteil	gering	jimp	Ja (Modul)
21	Vignettierung/Beleuchtungsgradient	mittel	jimp	Ja (Modul)
22	Schattenanteil an Rändern	mittel	jimp	Ja (Modul)
24	Bildrahmen/Randbeschnitt vorhanden	gering	jimp	Ja (Modul)
27	Bildrotation (0/90/180/270 grob)	mittel	jimp / Projektionsprofil	Ja (Modul)
29	Farbquantisierungs-Stufen	gering	jimp	Ja (Modul)
30	Salt-and-Pepper-Rauschanteil	mittel	jimp	Ja (Modul)
33	Farbkanal-Rauschverhältnis (SNR)	mittel	jimp	Ja (Modul)
35	Bit-Tiefe (8/16/32 Bit)	trivial	sharp	Ja (Modul)
36	Dateiformat-Kompressionstyp (verlustfrei/-behaftet)	trivial	sharp / file-type	Ja (Modul)
40	Symmetrie/Spiegelabweichung (Formulare)	mittel	jimp	Ja (Modul)
41	Hintergrundfarb-Uniformität	mittel	jimp	Ja (Modul)
45	Spaltenanzahl-Schätzung (vertikale Lücken)	mittel	jimp (Projektion)	Ja (Modul)
46	Textzeilen-Anzahl-Schätzung (horiz. Projektion)	mittel	jimp (eigen)	Ja (Modul)
47	Zeilenabstand-Konsistenz (Projektionsprofil-Peaks)	mittel	jimp (eigen)	Ja (Modul)
49	Verhältnis Vorder-/Hintergrundfläche	gering-mittel	jimp	Ja (Modul)
50	Bildkanten-Beschnitt-Verlust (Randobjekte abgeschnitten)	mittel	jimp	Ja (Modul)
51	Zeichenhöhe in Pixel (x-Height)	mittel	jimp (Projektionsprofil)	Ja (Modul)
58	Upsampling-/Interpolations-Erkennung	mittel	jimp	Ja (Modul)

Ausgeschlossen (zu hoher Aufwand oder komplexes Modul)

#	MESSGRÖSSE	AUFWAND	LIBRARY	GRUND
3	Schärfe (Laplace-Varianz)	mittel	opencv4nodejs	komplexes Modul
6	Schräglage/Skew-Winkel (Hough)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
8	Kantendichte (Sobel-Gradient)	mittel-hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
23	Glanzlicht/Specular-Highlight-Fläche	mittel	opencv4nodejs	komplexes Modul
25	Seitenwölbung (Buchscan-Krümmung)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
26	Perspektivische Verzerrung	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
28	Moiré-Muster-Stärke (FFT-Peak)	hoch	fft-js	hoher Aufwand + komplexes

				Modul
31	Bewegungsunschärfe (Richtungs-Gradient)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
32	Gaussian-Blur-Schätzwert	mittel	opencv4nodejs	komplexes Modul
34	Chromatische Aberration (Kantenversatz RGB)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
37	JPEG-Qualitätsschätzung (DCT-Analyse)	hoch	jpeg-js	hoher Aufwand + komplexes Modul
38	Lokale Kontrastvarianz (Fensterweise)	mittel= hoch	jimp	hoher Aufwand
39	Texturkomplexität (GLCM-Homogenität)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
42	Staub-/Kratzer-Erkennung (isolierte Ausreisser)	mittel= hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
43	Wasserzeichen-/Stempelüberlagerung	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
44	Tabellen-/Linienstruktur (Hough h/v)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
48	Connected-Components-Anzahl (Layout-Grobmass)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
52	Text-vs-Nichttext-Klassifikation (Textflächenanteil)	mittel	opencv4nodejs	komplexes Modul
53	Handschrift- vs. Druckschrift-Indikator	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
54	Lokale Schärfe-Varianz (Fokus-Ungleichmässigkeit)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
55	Glanzlicht-/Reflexionsflächen (Glare)	mittel	opencv4nodejs	komplexes Modul
56	Durchscheinen der Rückseite (Show-through)	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
57	Strichstärke/Stroke-Width-Konsistenz	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
59	Barcode-/Logo-/Stempel-Overlay auf Textbereich	hoch	opencv4nodejs	hoher Aufwand + komplexes Modul
60	Mehrfach-Kompressionsgenerationen (JPEG-Ghosting)	hoch	jpeg-js	hoher Aufwand + komplexes Modul



Aufbau und Felder des Dashboards

Kopfbereich

- **Titel & Badge:** Kennzeichnet das Dashboard und den Bezug zu Kapitel 8.2 (Datenquelle: SCORE-DB und Zeitreihen-DB).

KPI-Kacheln (obere Zeile)

- **Gesamterkennungsrate (aktuell):** aktuellster Wert der Ansicht 1, mit Veränderung gegenüber der Vorperiode — gibt dem Administrator auf einen Blick den Gesamtzustand des Systems.
- **Aktive Verarbeitungsmodule:** Anzahl und Auflistung der aktuell im Eskalationsmechanismus (Kap. 6.2) berücksichtigten Module.
- **Modul mit Qualitätsrückgang:** automatisch identifiziertes Modul mit dem stärksten Rückgang der Erkennungsrate über den Betrachtungszeitraum — direkter Hinweis für einen möglichen Modulaustausch (Kap. 6.3).
- **Messgrösse mit sinkender Predictability:** automatisch identifizierte Messgrösse mit dem stärksten Rückgang des Gewichtungsfaktors — Hinweis auf möglichen Überarbeitungsbedarf des Messgrössen-Katalogs (Anhang A).
- **Datensätze in der SCORE-DB:** Gesamtanzahl der aktuell gespeicherten Verarbeitungsfälle, inkl. Zugang der laufenden Woche und Abgang durch die Housekeeping-Löschung (Kap. 8.1) — zeigt, ob die Vergleichsbasis gesund wächst oder durch die Retention zu stark schrumpft.
- **Ø-Alter der SCORE-DB-Datensätze:** durchschnittliches Alter aller Datensätze in Tagen — ein steigender Wert deutet auf eine überalternde Vergleichsbasis hin (z. B. weil weniger neue Belege verarbeitet werden), ein stark sinkender Wert auf eine womöglich zu aggressive Löschung älterer Datensätze.

Panel 1 — Gesamterkennungsrate ohne Nachbearbeitung im Zeitverlauf

Liniendiagramm mit einer Linie für die systemweite Erkennungsrate je Woche (x-Achse: Kalenderwoche, y-Achse: Erkennungsrate in %). Entspricht der SQL-Abfrage aus Kapitel 8.2, Ansicht 1.

Panel 2 — Qualität der einzelnen Module im Zeitverlauf

Liniendiagramm mit einer Linie je Verarbeitungsmodul (Legende rechts). Im Beispiel ist ersichtlich, dass die Erkennungsrate von «LLM klein» über zwölf Wochen kontinuierlich sinkt, während die übrigen Module stabil bleiben — das Dashboard weist über eine Hinweisbox explizit auf die Empfehlung hin, dieses Modul gemäss Onboarding-Prozess (Kap. 6.3, FA-21) zu ersetzen.

Panel 3 — Predictability der Messgrössen im Zeitverlauf

Liniendiagramm mit einer Linie je Messgrösse (Legende rechts), basierend auf den Gewichtungsfaktoren aus der Zeitreihen-DB (Kap. 5.2). Im Beispiel verliert die Messgrösse «Zeichenhöhe» über die Zeit deutlich an Gewicht — die Hinweisbox empfiehlt, diese Messgrösse zu überprüfen bzw. den Katalog in Anhang A entsprechend zu erweitern.

Hinweis: Predictability bezeichnet den statistischen Zusammenhang zwischen einer einzelnen Messgrösse und dem OCR-Erfolg, operationalisiert als Feature-Importance-Wert. Er wird wöchentlich vom Housekeeping-Prozess mittels eines klassischen ML-Klassifikationsmodells auf den tabellarischen Daten der SCORE-DB berechnet (kein LLM; empfohlen für die vorliegende Datenmenge: LightGBM, siehe Kapitel 8) und in der Zeitreihen-DB (Kap. 5.2) historisiert. Sinkt der Wert über mehrere Läufe deutlich, verliert die Messgrösse an Aussagekraft.

Funktion des Dashboards

Das Dashboard dient ausschliesslich dem Administrator von SCORE (vgl. Abgrenzung, Kapitel 2.1) als Entscheidungsunterstützung für zwei wiederkehrende Fragestellungen:

1. **Wann sollte ein Verarbeitungsmodul (LLM) ausgetauscht werden?** — beantwortet durch Panel 2 in Kombination mit der KPI-Kachel «Modul mit Qualitätsrückgang».
2. **Ist der bestehende Messgrössen-Katalog noch aussagekräftig?** — beantwortet durch Panel 3 in Kombination mit der KPI-Kachel «Messgrösse mit sinkender Predictability».

Panel 1 liefert dabei den übergeordneten Gesamtkontext, vor dessen Hintergrund die beiden anderen Panels interpretiert werden: Ein Rückgang der Gesamterkennungsrate bei gleichzeitig stabilen Einzelmodulen deutet eher auf eine veränderte Belegpopulation hin (→ Panel 3 prüfen), während ein Rückgang bei einem einzelnen Modul auf ein Anbieter-/Modellproblem hindeutet (→ Panel 2 prüfen).

Es handelt sich um ein reines Mockup mit Beispieldaten zur Veranschaulichung; die tatsächliche technische Umsetzung (Visualisierungstechnologie, Aktualisierungsfrequenz, Zugriffsschutz) ist gemäss Kapitel 10 (offene Punkte) noch zu spezifizieren. Die HTML-Datei dieses Mockups (`SCORE_Dashboard_Mockup.html`) ist dieser PDF-Datei als Anhang beigelegt und kann direkt im Browser geöffnet werden.

Anhang C: DuckDB-Schema (Datenmodell)

Dieser Anhang enthält die vollständige, lauffähige SQL-Definition des in Kapitel 5 beschriebenen Datenmodells als DuckDB-Schema. Es umfasst die beiden Tabellen `score_db` (Fallspeicher, Kapitel 5.1) und `zeitreihen_db` (append-only geführter Gewichtungs- und Kostenverlauf, Kapitel 5.2), die View `v_zeitreihen_aktuell` für die jeweils aktuellsten Gewichte, die Fuzzy-Distanz-View `v_fuzzy_distanz` mit der gewichteten Distanzberechnung gemäss Kapitel 7 (die Gewichte je Messgrösse werden per Subquery aus der Zeitreihen-Tabelle bezogen), die Trigger-Ersatzmechanik für die Zeitstempel bei Datensatz-Anlage und Statusänderung, die Export-View `v_score_db_export` mit 60 inhaltlich benannten und ausführlich kommentierten Messwert-Spalten, das Benutzer- und Rechtekonzept (Least Privilege, Kapitel 8.3) sowie die Testdaten-Prozedur `seed_score_testdaten()`.

Hinweis: DuckDB unterstützt (Stand 2026) weder `CREATE TRIGGER` noch `CREATE PROCEDURE`; die entsprechenden Anforderungen sind DuckDB-idiomatisch über Spalten-`DEFAULT now()`, ein `update_status()`-Makro und ein Tabellen-Makro umgesetzt. Der äquivalente echte Trigger-Code für eine spätere Portierung auf den PostgreSQL-Cluster (Kapitel 13.2) ist im Schema als Kommentar enthalten. Die SQL-Datei (`score_duckdb_schema.sql`) ist dieser PDF zusätzlich als Anhang beigelegt.

```
-- =====
-- SCORE – DuckDB-Schema
-- =====
-- Enthält:
-- 1. Sequenzen für Primärschlüssel
-- 2. Tabelle score_db (Fallspeicher, Kapitel 5.1)
-- 3. Tabelle zeitreihen_db (Gewichtungs-/Kostenverlauf, Kapitel 5.2)
-- 4. View v_zeitreihen_aktuell (neueste Gewichte je Messgrösse/Modul)
-- 5. View v_fuzzy_distanz (gewichtete Distanzberechnung, Kapitel 7)
-- 6. Trigger-Ersatzmechanik (Zeitstempel bei INSERT / Status-UPDATE)
-- 7. View v_score_db_export (60 inhaltlich benannte Messwert-Spalten)
-- 8. DB-Benutzer & Zugriffsrechte (Least Privilege, Kapitel 8.3)
-- 9. seed_score_testdaten() (Prozedur/Makro zum Anlegen von Testdaten)
--
-- WICHTIGER HINWEIS ZU TRIGGERN
-- =====
-- DuckDB unterstützt (Stand 2026) KEINE CREATE TRIGGER-Syntax. Die beiden
-- geforderten Trigger werden daher DuckDB-idiomatisch umgesetzt:
-- * "on insert timestamp des Datensatzes" -> Spalten-DEFAULT now()
--   (feuert automatisch bei jedem INSERT, exakt wie ein BEFORE-INSERT-Trigger).
-- * "on update status timestamp setzen" -> die Statusänderung erfolgt
--   ausschliesslich ueber die vordefinierte UPDATE-Anweisung / das Makro
--   update_status(), das zeitstempel_statusaenderung = now() mitsetzt.
-- Zusätzlich ist unten der äquivalente CREATE-TRIGGER-Code als Kommentar
-- hinterlegt, falls das Schema spaeter auf eine Trigger-faehige Engine
-- (z. B. PostgreSQL, siehe Kapitel 13.2) portiert wird.
-- =====

-- =====
-- 1. SEQUENZEN (fuer Autoincrement-Primaerschluessel)
-- =====
CREATE SEQUENCE IF NOT EXISTS seq_score_db_pk START 1;
CREATE SEQUENCE IF NOT EXISTS seq_zeitreihen_pk START 1;

-- =====
-- 2. TABELLE score_db (Fallspeicher, Kapitel 5.1)
-- =====
CREATE TABLE IF NOT EXISTS score_db (
-- Primaerschluessel: automatisch generiert
pk BIGINT PRIMARY KEY
DEFAULT nextval('seq_score_db_pk'),

-- Zeitstempel des Datensatzes: wird per INSERT-DEFAULT gesetzt
-- (entspricht "Trigger on insert" -> setzt Timestamp des Datensatzes selbst)
zeitstempel TIMESTAMP NOT NULL DEFAULT now(),

-- Messgroessen-Vektor: 60 auf 0..100 normierte Werte.
-- Als DOUBLE[]-Array modelliert; Index 1..60 entspricht Messgroesse-ID 1..60.
messgroessen_vektor DOUBLE[] NOT NULL,

-- Verwendetes Verarbeitungsmodul (freier Modulname, nicht auf Werteliste beschraenkt)
```

```

verwendetes_modul          VARCHAR          NOT NULL,

-- Tokenverbrauch des Verarbeitungslaufs
token_anzahl              BIGINT            DEFAULT 0,

-- Kostenfaktor und daraus berechnete Gesamtkosten
kostenfaktor              DOUBLE            DEFAULT 0,
kosten_gesamt             DOUBLE            DEFAULT 0,

-- Status der Verarbeitung
status                    VARCHAR          NOT NULL DEFAULT 'in Bearbeitung'
                        CHECK (status IN (
                                'in Bearbeitung',
                                'erfolgreich',
                                'manuelle Nachbearbeitung erforderlich'
                                )),

-- Zeitpunkt der letzten Statusänderung.
-- Initial = Erstellzeit; wird bei jeder Statusänderung neu gesetzt
-- (entspricht "Trigger on update status").
zeitstempel_statusänderung  TIMESTAMP      NOT NULL DEFAULT now(),

-- Referenz auf ein Folgesystem (z. B. DATEV-Belegnummer)
referenz_folgesystem      VARCHAR,

-- Gewichteter Ähnlichkeitswert zum verwendeten Referenzdatensatz (Nachvollziehbarkeit)
fuzzy_distanz             DOUBLE,

-- Erkannte Felder + Log-Informationen des Verarbeitungslaufs (JSON)
daten                     JSON
);

-- =====
-- 3. TABELLE zeitreihen_db (Gewichtungs-/Kostenverlauf, Kapitel 5.2)
-- =====
-- APPEND-ONLY: jeder Housekeeping-Lauf schreibt je Messgroesse und Modul
-- einen neuen, mit Zeitstempel versehenen Datensatz (Historisierung fuer
-- die Predictability-Ansicht, Kapitel 8.2). Fuer die operative Nutzung
-- wird stets der jeweils AKTUELLSTE Datensatz je (messgroesse_id, modul)
-- herangezogen (siehe View v_zeitreihen_aktuell unten).
-- =====
CREATE TABLE IF NOT EXISTS zeitreihen_db (
    pk                      BIGINT          PRIMARY KEY
                        DEFAULT nextval('seq_zeitreihen_pk'),

    -- Referenz auf eine der 60 Messgroessen (1..60)
    messgroesse_id         INTEGER          NOT NULL
                        CHECK (messgroesse_id BETWEEN 1 AND 60),

    -- Verarbeitungsmodul (Gewichte gelten je Modul separat)
    verarbeitungsmodul     VARCHAR          NOT NULL,

    -- Vom Housekeeping-Prozess ermittelter Gewichtungsfaktor w_i (= Predictability)
    gewichtungsfaktor_w    DOUBLE          NOT NULL DEFAULT 1.0,

    -- Aktuell gueltiger Umrechnungsfaktor Token -> Kosten fuer dieses Modul
    kostenfaktor_je_token  DOUBLE          DEFAULT 0,

    -- Zeitpunkt des Housekeeping-Laufs, der diesen Datensatz geschrieben hat
    -- (per INSERT-DEFAULT gesetzt)
    zeitstempel_letzte_aktualisierung  TIMESTAMP NOT NULL DEFAULT now()
);

-- =====
-- 4a. HILFS-VIEW: jeweils AKTUELLSTE Gewichte je Messgroesse/Modul
-- =====
-- Aus der append-only-Zeitreihe wird per Fensterfunktion der neueste
-- Datensatz je (messgroesse_id, verarbeitungsmodul) selektiert.
-- =====
CREATE OR REPLACE VIEW v_zeitreihen_aktuell AS
SELECT
    messgroesse_id,
    verarbeitungsmodul,
    gewichtungsfaktor_w,
    kostenfaktor_je_token,
    zeitstempel_letzte_aktualisierung
FROM (
    SELECT
        z.*,
        ROW_NUMBER() OVER (
            PARTITION BY messgroesse_id, verarbeitungsmodul

```

```

        ORDER BY zeitstempel_letzte_aktualisierung DESC
    ) AS rn
    FROM zeitreihen_db z
) t
WHERE rn = 1;

-- =====
-- 4b. FUZZY-VIEW: gewichtete Distanzberechnung (Kapitel 7)
-- =====
-- Berechnet fuer JEDEN erfolgreichen Referenzdatensatz der score_db die
-- gewichtete quadrierte euklidische Distanz zu einem Anfragevektor:
--
--      Distanz = SUM_i [ w_i * (m_referenz,i - m_anfrage,i)^2 ]
--
-- Die Gewichte w_i stammen je Messgroesse aus der Zeitreihen-Tabelle
-- (Subquery auf v_zeitreihen_aktuell, passend zum Modul des Referenz-
-- datensatzes). Der Anfragevektor wird ueber die Parametertabelle
-- fuzzy_anfrage bereitgestellt (genau eine Zeile, Vektor der aktuellen Datei).
--
-- Nutzung im Hauptprozess:
-- 1. fuzzy_anfrage mit dem aktuellen Messgroessen-Vektor befuellen,
-- 2. SELECT ... FROM v_fuzzy_distanz LIMIT 1 (nur der aehnlichste Datensatz).
-- Es wird genau EIN Datensatz zurueckgeliefert; dessen verwendetes_modul
-- ist der Verfahrensvorschlag. (Frueher Top-3 mit anschliessender
-- Kostenauswahl; die Kostenoptimierung erfolgt nun ueber die
-- Eskalationslogik, Kapitel 6.2.)
-- =====

-- Parametertabelle fuer den Anfragevektor (genau eine aktive Zeile)
CREATE TABLE IF NOT EXISTS fuzzy_anfrage (
    id                INTEGER PRIMARY KEY DEFAULT 1,
    messgroessen_vektor DOUBLE[] NOT NULL,
    CHECK (id = 1)    -- erzwingt Singleton
);

CREATE OR REPLACE VIEW v_fuzzy_distanz AS
WITH anfrage AS (
    -- Anfragevektor in (Index, Wert) entpacken: Index 1..60
    SELECT
        idx                AS messgroesse_id,
        messgroessen_vektor[idx] AS wert_anfrage
    FROM fuzzy_anfrage,
        UNNEST(range(1, 61)) AS u(idx)    -- range(1,61) -> 1..60
),
referenz AS (
    -- Nur erfolgreiche Referenzdatensaetze ohne manuelle Nachbearbeitung
    SELECT
        s.pk,
        s.verwendetes_modul,
        s.kosten_gesamt,
        idx                AS messgroesse_id,
        s.messgroessen_vektor[idx] AS wert_referenz
    FROM score_db s,
        UNNEST(range(1, 61)) AS u(idx)
    WHERE s.status = 'erfolgreich'
),
gewichtete_differenzen AS (
    SELECT
        r.pk,
        r.verwendetes_modul,
        r.kosten_gesamt,
        -- Gewicht w_i je Messgroesse und Modul als Subquery aus der Zeitreihe;
        -- fehlt ein Wert, wird neutral mit 1.0 gewichtet (COALESCE).
        COALESCE(
            SELECT za.gewichtungsfaktor_w
            FROM v_zeitreihen_aktuell za
            WHERE za.messgroesse_id = r.messgroesse_id
            AND za.verarbeitungsmodul = r.verwendetes_modul
        ), 1.0)
    * POWER(r.wert_referenz - a.wert_anfrage, 2) AS gew_quadrat_diff
    FROM referenz r
    JOIN anfrage a
    ON a.messgroesse_id = r.messgroesse_id
)
SELECT
    pk,
    verwendetes_modul,
    kosten_gesamt,
    SUM(gew_quadrat_diff)                AS distanz
FROM gewichtete_differenzen
GROUP BY pk, verwendetes_modul, kosten_gesamt
ORDER BY distanz ASC;

```

```

-- =====
-- 5. TRIGGER-ERSATZMECHANIK
-- =====
-- 5a. INSERT-Zeitstempel:
--   Bereits ueber Spalten-DEFAULT now() geloest (siehe Tabellendefinition).
--   Ein INSERT ohne Angabe von zeitstempel / zeitstempel_letzte_aktualisierung
--   setzt den Timestamp automatisch:
--
--   INSERT INTO score_db (messgroessen_vektor, verwendetes_modul)
--   VALUES ([/* 60 Werte */], 'Low-Cost-OCR');
--   -> zeitstempel und zeitstempel_statusaenderung = now()

-- 5b. Status-UPDATE-Zeitstempel:
--   Da DuckDB keine Trigger kennt, erfolgt jede Statusaenderung ueber ein
--   Makro, das den Aenderungs-Zeitstempel zwingend mitsetzt. Dadurch kann
--   der Status nicht ohne Aktualisierung von zeitstempel_statusaenderung
--   geaendert werden.
CREATE OR REPLACE MACRO update_status(p_pk, p_neuer_status) AS TABLE
  UPDATE score_db
    SET status              = p_neuer_status,
        zeitstempel_statusaenderung = now()
  WHERE pk = p_pk;

--   Aufruf-Beispiel:
--   FROM update_status(8841, 'erfolgreich');
--
--   Alternativ direkt (immer beide Felder zusammen setzen):
--   UPDATE score_db
--     SET status = 'manuelle Nachbearbeitung erforderlich',
--         zeitstempel_statusaenderung = now()
--   WHERE pk = 8841;

-- =====
-- 5c. AEQUIVALENTE ECHTE TRIGGER (nur fuer Portierung auf PostgreSQL,
--   Kapitel 13.2 – in DuckDB auskommentiert lassen!)
-- =====
-- CREATE OR REPLACE FUNCTION trg_score_insert() RETURNS trigger AS $$
-- BEGIN
--   NEW.zeitstempel              := now();
--   NEW.zeitstempel_statusaenderung := now();
--   RETURN NEW;
-- END; $$ LANGUAGE plpgsql;

-- CREATE TRIGGER score_before_insert
-- BEFORE INSERT ON score_db
-- FOR EACH ROW EXECUTE FUNCTION trg_score_insert();

-- CREATE OR REPLACE FUNCTION trg_score_status_update() RETURNS trigger AS $$
-- BEGIN
--   IF NEW.status IS DISTINCT FROM OLD.status THEN
--     NEW.zeitstempel_statusaenderung := now();
--   END IF;
--   RETURN NEW;
-- END; $$ LANGUAGE plpgsql;

-- CREATE TRIGGER score_before_update_status
-- BEFORE UPDATE OF status ON score_db
-- FOR EACH ROW EXECUTE FUNCTION trg_score_status_update();
-- =====

-- =====
-- 6. EXPORT-VIEW: score_db mit inhaltlich benannten Messwert-Spalten
-- =====
-- Entpackt den messgroessen_vektor (DOUBLE[]) in 60 einzeln benannte
-- Spalten (snake_case gemaess DuckDB-Namenskonvention). Dient dem
-- Export/der Analyse (z. B. ML-Training, BI-Tools), wo sprechende
-- Spaltennamen statt Array-Indizes benoetigt werden.
-- Jede Spalte ist unten ausfuehrlich kommentiert.
-- =====
CREATE OR REPLACE VIEW v_score_db_export AS
SELECT
  pk,
  zeitstempel,
  verwendetes_modul,
  token_anzahl,
  kostenfaktor,
  kosten_gesamt,
  status,
  zeitstempel_statusaenderung,

```

```

referenz_folgesystem,
fuzzy_distanz,
-- --- 60 Messwerte (Index im Array 1..60) ---
messgroessen_vektor[1] AS bildaufloesung_ppi, -- Messgroesse 1: Bildauflösung relativ
zur Zeichenhöhe (PPI). Misst, wie fein der Beleg abgetastet wurde; zu geringe Auflösung führt
zu verwachsenen Zeichen und OCR-Fehlern.
messgroessen_vektor[2] AS kontrast_rms, -- Messgroesse 2: RMS-Kontrast:
Standardabweichung der Helligkeitswerte. Niedriger Kontrast (blasse Schrift auf hellem Grund)
erschwert die Zeichen-Hintergrund-Trennung.
messgroessen_vektor[3] AS schaefer_laplace_varianz, -- Messgroesse 3: Schärfe gemessen
als Varianz des Laplace-Operators. Niedrige Werte deuten auf Unschärfe/Defokus hin, die
Kanteninformation der Zeichen zerstört.
messgroessen_vektor[4] AS belichtung_histogramm_clipping, -- Messgroesse 4: Belichtung
als Anteil abgeschnittener Histogramm-Extreme (Clipping). Über-/Unterbelichtung lässt Zeichen
in Weiss- oder Schwarzflächen verschwinden.
messgroessen_vektor[5] AS bildrauschen_std_homogen, -- Messgroesse 5: Bildrauschen als
Standardabweichung in homogenen Flächen. Hohes Rauschen erzeugt Störpixel, die als falsche
Zeichenbestandteile interpretiert werden.
messgroessen_vektor[6] AS schraeglage_skew_hough, -- Messgroesse 6: Schräglage/Skew-
Winkel via Hough-Transformation. Geneigte Textzeilen verschlechtern zeilenbasierte OCR-
Segmentierung.
messgroessen_vektor[7] AS otsu_trennschaerfe, -- Messgroesse 7: Otsu-Trennschärfe
(Inter-Klassen-Varianz). Misst, wie klar sich Vorder- von Hintergrund per Schwellwert trennen
lässt – Grundlage der Binarisierung.
messgroessen_vektor[8] AS kantendichte_sobel, -- Messgroesse 8: Kantendichte über Sobel-
Gradient. Indikator für die Menge an Zeichenkonturen; sehr niedrig = kaum Text, sehr hoch =
mögliche Störstrukturen.
messgroessen_vektor[9] AS ink_coverage, -- Messgroesse 9: Tinten-Flächenanteil (Ink
Coverage): Anteil dunkler Pixel. Zu wenig = leerer Beleg, zu viel = überfüllt/verschmiert.
messgroessen_vektor[10] AS kompressionsartefakte_blockiness, -- Messgroesse 10:
Kompressionsartefakte als 8x8-Blockiness. JPEG-Blockstrukturen verfälschen feine
Zeichenkonturen.
messgroessen_vektor[11] AS farbtiefe_kanalanzahl, -- Messgroesse 11: Farbtiefe bzw.
Kanalanzahl (Graustufe vs. Farbe). Beeinflusst, welche Vorverarbeitung sinnvoll ist.
messgroessen_vektor[12] AS byte_pro_pixel, -- Messgroesse 12: Byte pro Pixel als Mass
der effektiven Kompressionsrate. Sehr niedrige Werte deuten auf starke, verlustbehaftete
Kompression hin.
messgroessen_vektor[13] AS seitenverhaeltnis_plausibilitaet, -- Messgroesse 13:
Plausibilität des Seitenverhältnisses. Ungewöhnliche Verhältnisse weisen auf Beschnitt oder
falsche Ausrichtung hin.
messgroessen_vektor[14] AS bildgroesse_pixelzahl, -- Messgroesse 14: Absolute Bildgröße
(Gesamtpixelzahl). Zu kleine Bilder tragen zu wenig Information für zuverlässige Erkennung.
messgroessen_vektor[15] AS farbstich_cast, -- Messgroesse 15: Farbstich/Cast als
Kanalabweichung. Starker Farbstich (z. B. Gelbstich alter Scans) kann Binarisierung stören.
messgroessen_vektor[16] AS saettigung_mittel, -- Messgroesse 16: Mittlere Farbsättigung.
Hilft, farbige Störelemente von neutralem Text zu unterscheiden.
messgroessen_vektor[17] AS dynamikumfang_spannweite, -- Messgroesse 17: Dynamikumfang
als Histogramm-Spannweite. Geringe Spannweite = flaes Bild mit schlechter Zeichenabgrenzung.
messgroessen_vektor[18] AS bild_entropie, -- Messgroesse 18: Bild-Entropie als Mass des
Informationsgehalts. Sehr niedrige Entropie deutet auf leere/uniforme Belege hin.
messgroessen_vektor[19] AS histogramm_peakanzahl, -- Messgroesse 19: Anzahl Histogramm-
Peaks (Bimodalität). Klar bimodale Verteilung begünstigt saubere Schwarz-Weiss-Trennung.
messgroessen_vektor[20] AS weissraum_anteil, -- Messgroesse 20: Weissraum-Anteil. Sehr
hoher Anteil = wenig Text; sehr niedriger = dichte, schwer trennbare Layouts.
messgroessen_vektor[21] AS vignettierung_gradient, -- Messgroesse 21:
Vignettierung/Beleuchtungsgradient. Ungleichmässige Ausleuchtung erschwert globale
Schwellwertverfahren.
messgroessen_vektor[22] AS schatten_raender, -- Messgroesse 22: Schattenanteil an den
Rändern. Typisch bei Handy-Fotos; Randschatten können Zeichen verdecken.
messgroessen_vektor[23] AS glanzlicht_specular_flaeche, -- Messgroesse 23: Glanzlicht-
/Specular-Highlight-Fläche. Reflexionen (z. B. auf Thermopapier) löschen lokal
Zeicheninformation.
messgroessen_vektor[24] AS randbeschnitt_vorhanden, -- Messgroesse 24:
Bildrahmen/Randbeschnitt vorhanden. Zeigt an, ob der Beleg vollständig erfasst oder
angeschnitten wurde.
messgroessen_vektor[25] AS seitenwoelbung_kruemmung, -- Messgroesse 25: Seitenwölbung
(Buchscan-Krümmung). Gekrümmte Zeilen verzerren Zeichen und Zeilenverlauf.
messgroessen_vektor[26] AS perspektivische_verzerrung, -- Messgroesse 26:
Perspektivische Verzerrung. Schräg fotografierte Belege verzerren Zeichenproportionen.
messgroessen_vektor[27] AS bildrotation_grob, -- Messgroesse 27: Grobe Bildrotation
(0/90/180/270°). Falsche Orientierung macht OCR ohne Vorab-Drehung unmöglich.
messgroessen_vektor[28] AS moire_fft_peak, -- Messgroesse 28: Moiré-Muster-Stärke via
FFT-Peak. Interferenzmuster (Rescan von Rasterdruck) überlagern Zeichen.
messgroessen_vektor[29] AS farbquantisierung_stufen, -- Messgroesse 29:
Farbquantisierungs-Stufen. Starke Quantisierung (wenige Farben) kann feine Kanten zerstören.
messgroessen_vektor[30] AS salt_pepper_anteil, -- Messgroesse 30: Salt-and-Pepper-
Rauschanteil. Isolierte schwarze/weiße Störpixel werden leicht als Zeichenteile
fehlinterpretiert.
messgroessen_vektor[31] AS bewegungsunschaerfe_richtung, -- Messgroesse 31:
Bewegungsunschärfe (gerichteter Gradient). Verwischte Zeichen durch Kamerabewegung.
messgroessen_vektor[32] AS gaussian_blur_schaetzwert, -- Messgroesse 32: Geschätzter
Gaussian-Blur-Grad. Allgemeine Weichzeichnung, die Kanten aufweicht.
messgroessen_vektor[33] AS farbkanal_snr, -- Messgroesse 33: Farbkanal-Rauschverhältnis

```

```

(SNR). Niedriges Signal-Rausch-Verhältnis mindert Erkennbarkeit.
    messgroessen_vektor[34] AS chromatische_aberration, -- Messgroesse 34: Chromatische
Aberration (RGB-Kantenversatz). Farbsäume an Kanten verwischen Zeichenkonturen.
    messgroessen_vektor[35] AS bit_tiefe, -- Messgroesse 35: Bit-Tiefe (8/16/32 Bit).
Beeinflusst die verfügbare Tonwertauflösung.
    messgroessen_vektor[36] AS kompressionstyp, -- Messgroesse 36: Dateiformat-
Kompressionstyp (verlustfrei vs. verlustbehaftet). Verlustbehaftung korreliert mit
Artefakten.
    messgroessen_vektor[37] AS jpeg_qualitaet_dct, -- Messgroesse 37: JPEG-
Qualitätsschätzung via DCT-Analyse. Niedrige geschätzte Qualität = mehr Artefakte.
    messgroessen_vektor[38] AS lokale_kontrastvarianz, -- Messgroesse 38: Lokale
Kontrastvarianz (fensterweise). Ungleichmässiger Kontrast über das Bild hinweg.
    messgroessen_vektor[39] AS texturkomplexitaet_glcml, -- Messgroesse 39: Texturkomplexität
(GLCM-Homogenität). Unterscheidet glatte Hintergründe von strukturiertem Text.
    messgroessen_vektor[40] AS symmetrie_abweichung, -- Messgroesse 40: Symmetrie-
/Spiegelabweichung (bei Formularen). Misst Ausrichtungsfehler strukturierter Vorlagen.
    messgroessen_vektor[41] AS hintergrund_uniformitaet, -- Messgroesse 41: Hintergrundfarb-
Uniformität. Ungleichmässiger Hintergrund erschwert Text-Freistellung.
    messgroessen_vektor[42] AS staub_kratzer, -- Messgroesse 42: Staub-/Kratzer-Erkennung
(isolierte Ausreisser). Physische Defekte auf Scans.
    messgroessen_vektor[43] AS wasserzeichen_stempel, -- Messgroesse 43: Wasserzeichen-
/Stempelüberlagerung. Halbtransparente Überlagerungen stören Zeichen.
    messgroessen_vektor[44] AS tabellen_linienstruktur, -- Messgroesse 44: Tabellen-
/Linienstruktur (Hough horizontal/vertikal). Erkennt Formular- und Tabellenlinien.
    messgroessen_vektor[45] AS spaltenanzahl_schaetzung, -- Messgroesse 45: Spaltenanzahl-
Schätzung (vertikale Lücken). Mehrspaltige Layouts erfordern besondere Segmentierung.
    messgroessen_vektor[46] AS textzeilen_anzahl_schaetzung, -- Messgroesse 46: Textzeilen-
Anzahl-Schätzung (horizontale Projektion). Grobmass für die Textmenge.
    messgroessen_vektor[47] AS zeilenabstand_konsistenz, -- Messgroesse 47: Zeilenabstand-
Konsistenz (Projektionsprofil-Peaks). Unregelmässige Abstände erschweren Zeilentrennung.
    messgroessen_vektor[48] AS connected_components_anzahl, -- Messgroesse 48: Connected-
Components-Anzahl (Layout-Grobmass). Zahl zusammenhängender Objekte als
Komplexitätsindikator.
    messgroessen_vektor[49] AS vorder_hintergrund_verhaeltnis, -- Messgroesse 49: Verhältnis
Vorder-/Hintergrundfläche. Balance zwischen Text und Leerraum.
    messgroessen_vektor[50] AS randobjekt_beschnitt, -- Messgroesse 50: Bildkanten-
Beschnitt-Verlust (abgeschnittene Randobjekte). Zeigt verlorene Information am Bildrand an.
    messgroessen_vektor[51] AS zeichenhoehe_xheight, -- Messgroesse 51: Zeichenhöhe in Pixel
(x-Height). Zu kleine Zeichen unterschreiten die zuverlässige Erkennungsgrösse.
    messgroessen_vektor[52] AS textflaechenanteil, -- Messgroesse 52: Text-vs-Nichttext-
Klassifikation (Textflächenanteil). Anteil echter Textregionen am Bild.
    messgroessen_vektor[53] AS handschrift_indikator, -- Messgroesse 53: Handschrift- vs.
Druckschrift-Indikator. Handschrift erfordert andere Modelle als Maschinenschrift.
    messgroessen_vektor[54] AS lokale_schaerfe_varianz, -- Messgroesse 54: Lokale Schärfe-
Varianz (Fokus-Ungleichmässigkeit). Teilbereiche unterschiedlicher Schärfe.
    messgroessen_vektor[55] AS glare_reflexion, -- Messgroesse 55: Glanzlicht-
/Reflexionsflächen (Glare). Grossflächige Reflexionen, die Text auslöschen.
    messgroessen_vektor[56] AS show_through, -- Messgroesse 56: Durchscheinen der Rückseite
(Show-through). Rückseitentext scheint durch und stört die Erkennung.
    messgroessen_vektor[57] AS strichstaerke_konsistenz, -- Messgroesse 57:
Strichstärke/Stroke-Width-Konsistenz. Ungleichmässige Strichstärken deuten auf Druck-
/Scanprobleme hin.
    messgroessen_vektor[58] AS upsampling_erkennung, -- Messgroesse 58: Upsampling-
/Interpolations-Erkennung. Künstlich hochskalierte Bilder haben keine echte
Detailinformation.
    messgroessen_vektor[59] AS overlay_auf_text, -- Messgroesse 59: Barcode-/Logo-/Stempel-
Overlay auf Textbereich. Überlagerungen direkt über Text mindern Lesbarkeit.
    messgroessen_vektor[60] AS mehrfach_kompression_ghosting, -- Messgroesse 60: Mehrfach-
Kompressionsgenerationen (JPEG-Ghosting). Wiederholtes Speichern erzeugt kumulierte
Artefakte.
daten
FROM score_db;

-- =====
-- 7. DATENBANK-BENUTZER UND ZUGRIFFSRECHTE (Least Privilege, Kap. 8.3)
-- =====
-- Umsetzung des Least-Privilege-Prinzips: jeder Dienst erhaelt einen
-- eigenen Benutzer mit exakt den benoetigten Rechten.
--
-- HINWEIS ZUR PORTIERUNG: DuckDB besitzt seit v1.1 ein experimentelles
-- Rechte-/Secret-System, das je nach Deployment (Einzeldatei vs. Server)
-- unterschiedlich ausfaellt. In einer reinen Einzeldatei-DuckDB gibt es
-- keine klassische Nutzerverwaltung; die Rechtetrennung wird dann auf
-- Ebene des vorgelagerten PostgreSQL-Clusters (Kap. 13.2) durchgesetzt.
-- Die folgenden Anweisungen sind in ANSI-SQL/PostgreSQL-Syntax gehalten
-- und dienen als verbindliche Rechte-Spezifikation fuer den persistenten
-- Cluster; auf einer DuckDB-Serverinstanz mit aktiviertem Auth-Layer sind
-- sie sinnigmaess anwendbar.
-- =====

-- 7a. Rollen/Benutzer je Dienst
CREATE ROLE score_hauptprozess LOGIN PASSWORD '****'; -- liest+schreibt Faelle, KEIN DELETE
CREATE ROLE score_housekeeping LOGIN PASSWORD '****'; -- einziger Dienst mit DELETE-Recht

```

```

CREATE ROLE score_ml_service LOGIN PASSWORD '****'; -- liest score_db, schreibt
zeitreihen_db
CREATE ROLE dashboard_reader LOGIN PASSWORD '****'; -- ausschliesslich lesend (Views)

-- 7b. Rechte Hauptprozess: Faelle anlegen, lesen, Status aktualisieren
GRANT SELECT, INSERT, UPDATE ON score_db TO score_hauptprozess;
GRANT SELECT ON zeitreihen_db TO score_hauptprozess;
GRANT SELECT ON v_zeitreihen_aktuell TO score_hauptprozess;
GRANT SELECT, INSERT, UPDATE ON fuzzy_anfrage TO score_hauptprozess;
GRANT SELECT ON v_fuzzy_distanz TO score_hauptprozess;
GRANT USAGE ON SEQUENCE seq_score_db_pk TO score_hauptprozess;

-- 7c. Rechte Housekeeping: als EINZIGER Dienst darf loeschen (Retention, Kap. 8.1)
GRANT SELECT, INSERT, UPDATE, DELETE ON score_db TO score_housekeeping;
GRANT SELECT, INSERT ON zeitreihen_db TO score_housekeeping; -- append-only
GRANT USAGE ON SEQUENCE seq_zeitreihen_pk TO score_housekeeping;

-- 7d. Rechte ML-Service: liest Faelle, schreibt neue Gewichte (append-only)
GRANT SELECT ON score_db TO score_ml_service;
GRANT SELECT ON v_score_db_export TO score_ml_service;
GRANT SELECT, INSERT ON zeitreihen_db TO score_ml_service;
GRANT USAGE ON SEQUENCE seq_zeitreihen_pk TO score_ml_service;

-- 7e. Rechte Dashboard: ausschliesslich lesend auf definierte Views
GRANT SELECT ON v_zeitreihen_aktuell TO dashboard_reader;
GRANT SELECT ON v_score_db_export TO dashboard_reader;
-- (auf die Dashboard-Aggregat-Views v_dash_* aus Kapitel 8.2, sobald angelegt)

-- =====
-- 8. TESTDATEN-PROZEDUR
-- =====
-- Legt eine konfigurierbare Anzahl zufaelliger, aber plausibler
-- Testdatensaeetze in score_db an (Messwerte 0..100, zufaelliges Modul
-- und Status). Praktisch fuer Integrationstests (Kap. 14) und zum
-- Ausprobieren der Fuzzy-View.
--
-- DuckDB kennt keine CREATE PROCEDURE-Anweisung; eine parametrierbare
-- "Prozedur" wird daher als table-MACRO umgesetzt, das die INSERT-Logik
-- kapselt und per FROM aufgerufen wird.
-- =====
CREATE OR REPLACE MACRO seed_score_testdaten(p_anzahl) AS TABLE
  INSERT INTO score_db (
    messgroessen_vektor,
    verwendetes_modul,
    token_anzahl,
    kostenfaktor,
    kosten_gesamt,
    status,
    fuzzy_distanz,
    daten
  )
  SELECT
    -- 60 Zufallswerte 0..100 als DOUBLE[] erzeugen
    (SELECT list(round(random() * 100, 2))
     FROM range(1, 61)) AS messgroessen_vektor,
    -- zufaelliges Modul
    ([ 'Low-Cost-OCR', 'LLM klein', 'LLM mittel', 'LLM gross' ]
     ) [1 + CAST(floor(random() * 4) AS INTEGER)] AS verwendetes_modul,
    CAST(floor(random() * 4000) AS BIGINT) AS token_anzahl,
    round(random() * 0.02, 5) AS kostenfaktor,
    round(random() * 0.10, 5) AS kosten_gesamt,
    -- ~70% erfolgreich, Rest Nachbearbeitung
    CASE WHEN random() < 0.7
      THEN 'erfolgreich'
      ELSE 'manuelle Nachbearbeitung erforderlich'
    END AS status,
    round(random() * 50, 3) AS fuzzy_distanz,
    json_object('quelle', 'testdaten', 'lauf', i) AS daten
  FROM range(1, CAST(p_anzahl AS BIGINT) + 1) AS t(i);

-- Aufruf-Beispiele:
-- FROM seed_score_testdaten(100); -- 100 Testfaelle anlegen
-- SELECT count(*) FROM score_db; -- Kontrolle
--
-- Beispielhafter Fuzzy-Testlauf nach dem Seeding:
-- -- Anfragevektor setzen (60 Werte):
-- INSERT INTO fuzzy_anfrage (id, messgroessen_vektor)
-- VALUES (1, (SELECT list(round(random()*100,2)) FROM range(1,61)))
-- ON CONFLICT (id) DO UPDATE
-- SET messgroessen_vektor = EXCLUDED.messgroessen_vektor;
-- -- Aehnlichsten erfolgreichen Fall abfragen (genau ein Datensatz):
-- SELECT pk, verwendetes_modul, kosten_gesamt, distanz

```

```
--      FROM v_fuzzy_distanz
--      LIMIT 1;
-- =====
```

[↑ Zum Inhaltsverzeichnis](#)